

CREACIÓN DE INTERFACES EN C#

CFGS Desarrollo de Aplicaciones
Multiplataforma

Desarrollo de interfaces

Carmen Campos Fernández

CONTENIDO

Contenido	2
1. Delegados	4
1.1. ¿Qué son los delegados?	4
1.2. Diferencias con Java	4
1.3. Declaración de un Delegado.....	4
1.4. Asignación de Métodos a Delegados	4
1.5. Delegados Multicasting	5
1.6. Delegados y Funciones de <i>Callback</i>	6
2. Métodos anónimos.....	6
2.1. ¿Qué son los métodos anónimos?	6
2.2. ¿Para Qué Sirven los Métodos Anónimos?	6
2.3. Declaración y Uso de Métodos Anónimos	7
3. Eventos y delegados	8
3.1. ¿Qué son los eventos?.....	8
3.2. Ventajas del uso de eventos.....	8
3.3. ¿Cómo funcionan los eventos?	9
3.4. ¿Para qué sirven los eventos?	10
4. WPF (Windows Presentation Foundation)	21
4.1. ¿Qué es WPF?	21
4.2. ¿Qué es XAML?	21
4.3. Primer Proyecto: Hola mundo	22
4.4. StackPanel, ListBox y Árboles Visual y Lógico.....	25
4.5. Eventos RoutedEvents_3 Tipos: Direct, Bubbling y Tunneling.....	28
4.6. Grid	32
a. Manejo de excepciones.....	33
5. CONTROLES BÁSICOS.....	34
5.1. TEXTBLOCK.....	34
5.2. El control LABEL	42
5.3. El control TEXTBOX.....	45
5.4. El control BUTTON	48

5.5.	El control CHECKBOX	52
5.6.	Control RADIOBUTTON.....	56
5.7.	Control PASSWORDBOX	60
6.	Propiedades de dependencia	61
6.1.	DependencyProperty personalizadas.....	63
7.	Enlace de datos: DataBinding.....	64
7.1.	Binding con propiedades de dependencia personalizada.....	66
7.2.	El control LISTBOX	69
7.3.	El control COMBOBOX.....	73
8.	Tooltip: Información sobre herramientas	74
9.	MessageBox.....	75
9.1.	MESSAGEBOX con título	75
9.2.	MESSAGEBOX CON BOTONES EXTRA	76
9.3.	MESSAGEBOX con un icono.....	77
9.4.	MESSAGEBOX con una opción por defecto.....	78
10.	ANEXO I: Ejemplo completo de Evento en C#.....	80
11.	ANEXO II: Método Invoke	85

1. DELEGADOS

1.1. ¿Qué son los delegados?

En C#, un delegado es un tipo que representa referencias a métodos con una firma específica. Los delegados permiten encapsular un método y pasarlo como un argumento, similar a las interfaces funcionales en Java.

1.2. Diferencias con Java

En Java, se utilizan interfaces funcionales y expresiones lambda para lograr un comportamiento similar. En C#, los delegados son más explícitos y están más integrados en el lenguaje.

1.3. Declaración de un Delegado

Para declarar un delegado, se usa la palabra clave `delegate`, seguida de la firma del método que representa.

Ejemplo:

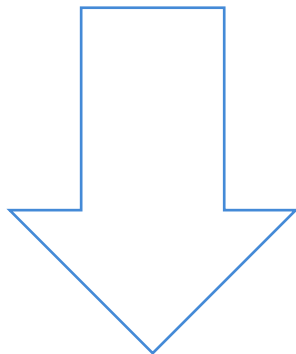
```
delegate int Operacion(int x, int y);
```

Este delegado puede apuntar a cualquier método que acepte dos enteros y devuelva un entero.

1.4. Asignación de Métodos a Delegados

Una vez que se ha declarado un delegado, se puede asignar métodos a él. Un delegado puede apuntar a un solo método o a múltiples métodos (delegados multicasting).

Ejemplo:



```

using System;

delegate int Operacion(int x, int y);

class Program
{
    static int Sumar(int a, int b) => a + b;
    static int Restar(int a, int b) => a - b;

    static void Main()
    {
        Operacion op = Sumar; // Asignar el método Sumar
        Console.WriteLine("Suma: " + op(5, 3)); // Salida: Suma: 8

        op = Restar; // Reasignar el método a Restar
        Console.WriteLine("Resta: " + op(5, 3)); // Salida: Resta: 2
    }
}

```

1.5. Delegados Multicasting

Los delegados pueden contener múltiples métodos. Cuando se llama al delegado, se ejecutan todos los métodos asignados en el orden en que fueron añadidos.

Ejemplo:

```

using System;

delegate int Operacion(int x, int y);

class Program
{
    static int Sumar(int a, int b) => a + b;
    static int Restar(int a, int b) => a - b;

    static void Main()
    {
        Operacion op = Sumar;
        op += Restar; // Añadir Restar

        foreach (Operacion metodo in op.GetInvocationList())
        {
            Console.WriteLine(metodo(5, 3)); // Salidas: 8 y 2
        }
    }
}

```

1.6. Delegados y Funciones de *Callback*

Los delegados son útiles para implementar funciones de *callback*, donde un método puede ser pasado como argumento a otro método.

```
using System;

delegate int OperacionCallback(int x, int y);

class Program
{
    static int Sumar(int a, int b) => a + b;

    static int EjecutarOperacion(OperacionCallback op, int x, int y)
    {
        return op(x, y);
    }

    static void Main()
    {
        Console.WriteLine("Suma: " + EjecutarOperacion(Sumar, 5, 3)); // Salida: Suma: 8
    }
}
```

Resumen

Los delegados son una herramienta poderosa en C# que permite el uso flexible de métodos como parámetros, facilitando patrones de diseño como el de observador o callbacks. Al entender cómo funcionan los delegados, los programadores pueden implementar una lógica más dinámica y reusable en sus aplicaciones.

2. MÉTODOS ANÓNIMOS

2.1. ¿Qué son los métodos anónimos?

En C#, un **método anónimo** es una manera de definir una función "al vuelo" sin necesidad de darle un nombre explícito. En lugar de crear un método separado con un nombre en el código, el método anónimo se define directamente en el lugar donde se necesita, lo que hace que el código sea más compacto y fácil de leer, especialmente en situaciones donde la función se usa solo una vez.

2.2. ¿Para Qué Sirven los Métodos Anónimos?

Los métodos anónimos son útiles cuando:

- La lógica es pequeña y solo se usará una vez.
- No se necesita crear un método separado, lo cual hace el código más legible y compacto.

2.3. Declaración y Uso de Métodos Anónimos

Un método anónimo en C# se declara usando la palabra clave `delegate`, seguida de los parámetros entre paréntesis y el cuerpo del método en `{ }`. Por ejemplo:

```
delegate(int x, int y) {
    return x + y;
};
```

Para utilizar un método anónimo, generalmente se asigna a un delegado o se pasa como parámetro a un método que espera un delegado.

EJEMPLO BÁSICO DE MÉTODO ANÓNIMO

1. Definir un delegado que acepte un método anónimo:

```
delegate int Operacion(int a, int b);
```

2. Asignar un método anónimo al delegado:

```
Operacion suma = delegate(int a, int b) {
    return a + b;
};
```

3. Usar el método anónimo:

```
int resultado = suma(5, 3); // Resultado: 8
```

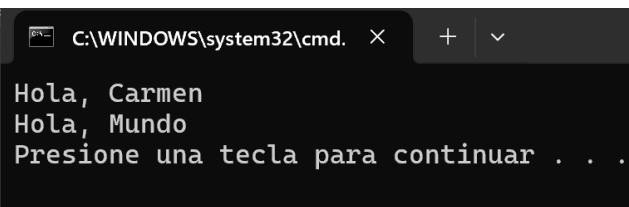
Ejemplo de uso:

```
namespace MetodosAnonimos
{
    0 referencias
    internal class Program
    {
        public delegate string ObteberDelegadoTexto(string nombre);
        0 referencias
        static void Main(string[] args)
        {
            ObteberDelegadoTexto miDelegado = delegate (string nombre)
            {
                return "Hola, " + nombre;
            };
            Console.WriteLine(miDelegado("Carmen"));
        }
    }
}
```

Ahora vamos a pasar como parámetro, un delegado que contiene un método anónimo:

```
namespace MetodosAnonimos
{
    0 referencias
    internal class Program
    {
        public delegate string ObteberDelegadoTexto(string nombre);
        0 referencias
        static void Main(string[] args)
        {
            ObteberDelegadoTexto miDelegado = delegate (string nombre)
            {
                return "Hola, " + nombre;
            };
            Console.WriteLine(miDelegado("Carmen"));
            Mostrar(miDelegado);
        }
        1 referencia
        static void Mostrar (ObteberDelegadoTexto miDelegado)
        {
            Console.WriteLine(miDelegado("Mundo"));
        }
    }
}
```

Obtendríamos el siguiente resultado por pantalla:



```
C:\WINDOWS\system32\cmd. X + v
Hola, Carmen
Hola, Mundo
Presione una tecla para continuar . . .
```

3. EVENTOS Y DELEGADOS

3.1. ¿Qué son los eventos?

los **eventos** son un mecanismo que permite a una clase o a un objeto **notificar a otras clases u objetos cuando ocurre una acción específica**. Son una parte fundamental de la programación orientada a eventos y facilitan la **comunicación entre objetos**, permitiendo que un objeto avise a otros objetos sobre ciertos "eventos" o sucesos.

Ejemplo: cuando se pulsa un botón.

3.2. Ventajas del uso de eventos

-  **Desacoplamiento:** Los eventos permiten una comunicación flexible entre objetos sin que necesiten conocerse directamente. Esto facilita la modularidad y el mantenimiento del código.
-  **Escalabilidad:** Los eventos permiten que varios objetos respondan a la misma acción de forma independiente, lo que es útil en sistemas grandes y escalables.
-  **Código más legible y organizado:** Los eventos permiten que las clases gestionen su propia lógica de notificación, manteniendo el código ordenado.
-  **Respuesta a acciones del usuario o sistema:** Los eventos facilitan responder a acciones como clics de usuario, cambios de estado, finalización de tareas, etc., lo cual es vital en aplicaciones interactivas.

3.3. ¿Cómo funcionan los eventos?

Un evento en C# requiere dos componentes:

1. **Delegado:** Define la firma del método que manejará el evento.

Ejemplo:

```
public delegate void AlarmaActivadaHandler(object sender, EventArgs e);
```

2. **Evento:**

- ✧ Se declara en la clase.
- ✧ **Utiliza el delegado como su tipo.**
- ✧ Otros objetos se suscriben al evento, y cuando éste se activa, los métodos suscritos se ejecutan automáticamente.

```

using System;

class Program
{
    // Delegado que define la firma del método para el evento
    public delegate void MiDelegadoEvento();

    // Evento basado en el delegado
    public static event MiDelegadoEvento MiEvento;

    static void Main()
    {
        // Suscripción al evento
        MiEvento += ManejadorEvento;

        // Activar el evento
        MiEvento?.Invoke();
    }

    // Método que maneja el evento
    static void ManejadorEvento()
    {
        Console.WriteLine("Evento manejado.");
    }
}

```

En este ejemplo, `MiEvento` es el evento, `MiDelegadoEvento` es el delegado, y `ManejadorEvento` es el método que responde al evento. Al activarse `MiEvento` (mediante `.Invoke()`), se ejecuta `ManejadorEvento`.

3.4. ¿Para qué sirven los eventos?

Los eventos son útiles en situaciones donde una clase necesita notificar a otras clases que algo ha sucedido. Ejemplos:

1. **Interfaz de usuario:** Los eventos son fundamentales en aplicaciones con interfaces gráficas (GUI), donde el usuario interactúa con botones, menús, o cualquier otro control.
 - Ejemplo: El evento Click de un botón.

2. **Monitoreo de cambios de estado:** Si un objeto necesita que otros sepan cuando cambia su estado, puede usar eventos.
 - Ejemplo: Un sensor que activa un evento cuando supera un umbral de temperatura.
3. **Comunicación entre componentes:** En aplicaciones grandes, los eventos permiten que los componentes interactúen de manera desacoplada.
 - Ejemplo: Una clase de "reloj" que dispara un evento cada segundo.

Ejemplo completo:

```
using System;
using System.Threading;

class Reloj
{
    public delegate void SegundoCambiadoHandler(object sender, EventArgs e);
    public event SegundoCambiadoHandler SegundoCambiado;

    public void Iniciar()
    {
        while (true)
        {
            Thread.Sleep(1000);
            OnSegundoCambiado();
        }
    }

    protected virtual void OnSegundoCambiado()
    {
        if (SegundoCambiado != null)
        {
            SegundoCambiado(this, EventArgs.Empty);
        }
    }
}

class Program
{
    static void Main()
    {
        Reloj reloj = new Reloj();
        reloj.SegundoCambiado += Reloj_SegundoCambiado;

        reloj.Iniciar();
    }

    static void Reloj_SegundoCambiado(object sender, EventArgs e)
    {
        Console.WriteLine("¡Ha pasado un segundo!");
    }
}
```

❖ ¿Quién lanza el evento aquí?

En este caso, **la clase Reloj** es la que lanza el evento.

Por eso, dentro de Reloj, hay el método:

```
protected virtual void OnSegundoCambiado()
{
    if (SegundoCambiado != null)
    {
        SegundoCambiado(this, EventArgs.Empty);
    }
}
```

❖ Aquí pasa lo siguiente:

1. SegundoCambiado es el evento (que puede tener suscriptores).
2. Si hay suscriptores (`!= null`), se llama a ese evento.
3. El evento, al ser llamado, ejecuta todos los métodos que se han suscrito a él.
4. Se pasa como parámetro `this`, que significa:
→ “el objeto que lanza el evento soy yo mismo” (el reloj que está corriendo).

❖ ¿Por qué existe el método `Iniciar()`?

El evento **no se lanza automáticamente**.

Debe ocurrir algo en el tiempo — por ejemplo, que pase un segundo.

Ese “algo” se simula con un bucle infinito en `Iniciar()`:

```
public void Iniciar()
{
    while (true)
    {
        Thread.Sleep(1000); // espera 1 segundo
        OnSegundoCambiado(); // lanza el evento
    }
}
```

Así que el flujo es:

1. `Iniciar()` corre para siempre.
2. Espera 1 segundo (`Thread.Sleep(1000)`).
3. Llama al método `OnSegundoCambiado()`, que dispara el evento.
4. Cualquier suscriptor recibe la notificación.

❖ ¿Y quién escucha ese evento?

La clase `Program`:

```

class Program
{
    static void Main()
    {
        Reloj reloj = new Reloj();

        // Aquí nos "suscribimos" al evento del reloj
        reloj.SegundoCambiado += Reloj_SegundoCambiado;

        reloj.Iniciar(); // esto empieza el ciclo de eventos
    }

    static void Reloj_SegundoCambiado(object sender, EventArgs e)
    {
        Console.WriteLine("¡Ha pasado un segundo!");
    }
}

```

Así que:

- *Program* crea el *Reloj*.
- Se **suscribe** a su evento *SegundoCambiado*.
- Cuando el *Reloj* lanza el evento cada segundo, el método *Reloj_SegundoCambiado* se ejecuta automáticamente.

Flujo completo:

```

Main() → crea reloj
|
├→ se suscribe al evento SegundoCambiado
|
├→ llama reloj.Iniciar()
|
└→ (dentro de Reloj)
    while(true)
        |
        ├── espera 1 segundo
        ├── llama OnSegundoCambiado()
        ├── evento se dispara
        └→ se ejecuta Reloj_SegundoCambiado() en Program

```

En resumen:

Orden real de ejecución

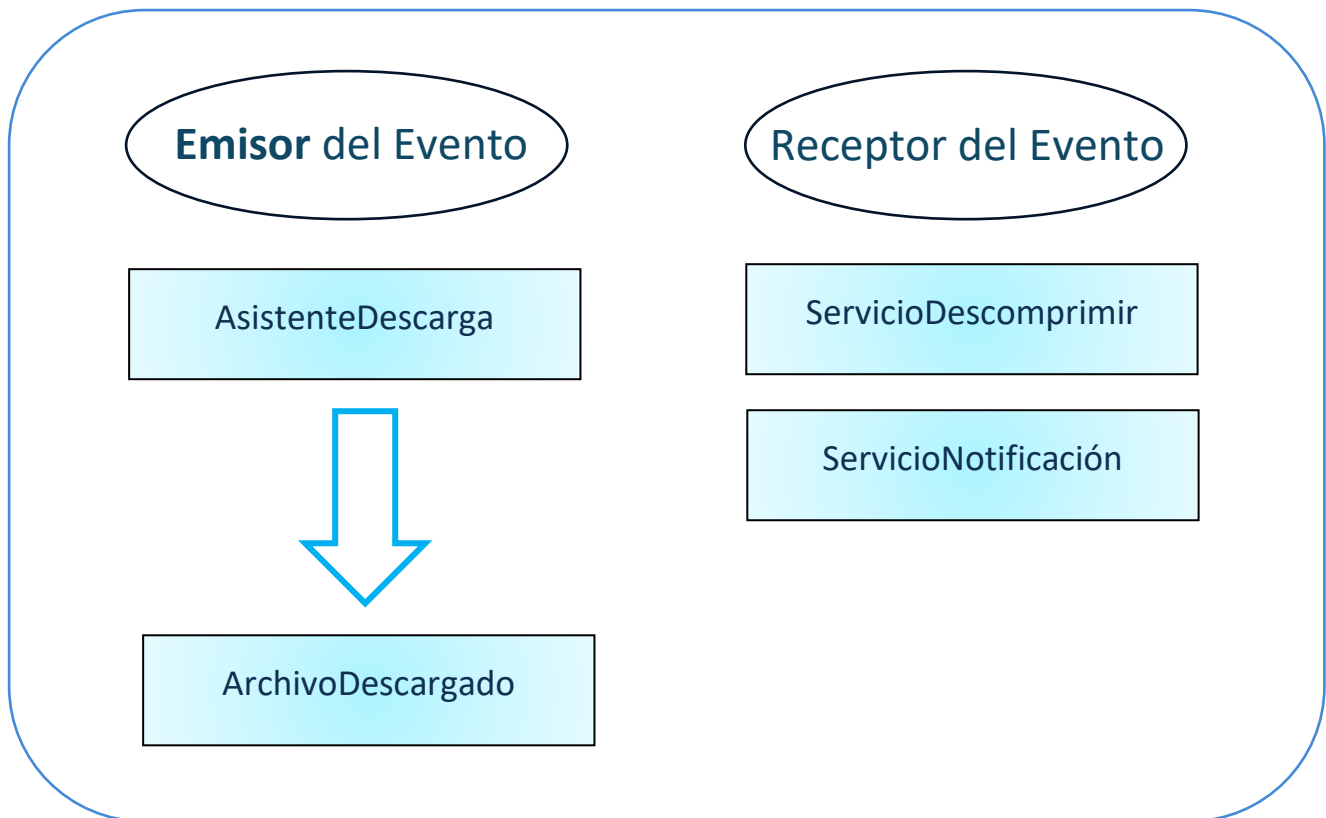
1. El programa arranca en Main().
2. Se crea el objeto reloj.
3. Se suscribe el método Reloj_SegundoCambiado al evento.
4. Se llama reloj.Iniciar().
5. Dentro de Iniciar(), comienza el bucle infinito:
 - Espera 1 segundo.
 - Llama OnSegundoCambiado().
 - Este dispara el evento SegundoCambiado.
6. Como Program está suscrito, se ejecuta su método:
 - Reloj_SegundoCambiado() imprime “**iHa pasado un segundo!**”.
7. Se repite para siempre (cada segundo).

Ejemplo mental

Imagina que Reloj es una **alarma**  y Program es una **persona escuchando** 

- La alarma (Reloj) tiene un botón interno (OnSegundoCambiado) que suena cada segundo.
- Cada vez que suena, “avisa” a quien esté escuchando (evento).
- La persona (Program) se suscribe diciendo:
“Cuando suene, ejecuta este método (Reloj_SegundoCambiado)”.

Ejemplo paso a paso



¿Cómo implementamos este flujo?

A través de **delegados**. En este caso los delegados:

- El Delegado es un contrato entre el emisor y el receptor.
- Determina la firma de un gestor de eventos en el receptor y puede contener muchos punteros de métodos.

Los hacemos en Visual Studio...

🛠 Creamos las estructuras básicas de las tres clases que usaremos, sin completar

1. Creamos la clase Archivo:

```

using System;

using System.Threading.Tasks;

namespace EventosyDelegados_v2
{
    1 referencia
    public class Archivo
    {
        0 referencias
        public string Titulo { get; set; }
    }
}

```

2. Creamos la clase AsistenteDescarga:

```

using System;

using System.Threading;

namespace EventosyDelegados_v2
{
    0 referencias
    public class AsistenteDescarga
    {
        //Método que simula la descarga esperando 4 segundos a través de un Thread.
        0 referencias
        public void Descarga(Archivo archivo)
        {
            Console.WriteLine("Descargando el archivo...");
            Thread.Sleep(4000);
        }
    }
}

```

3. Configuramos la base de la clase Program que contendrá el Main:

```

using System;

using System.Threading.Tasks;

namespace EventosyDelegados_v2
{
    0 referencias
    internal class Program
    {
        0 referencias
        static void Main(string[] args)
        {
            var archivo = new Archivo() { Titulo="Archivo 1"};
            var asistenteDescarga = new AsistenteDescarga();//emisor

            asistenteDescarga.Descargar(archivo);//Llamamos al método que lanza el evento
        }
    }
}

```


Completamos cada clase

Clase AsistenteDescarga:

- 1) Creamos un delegado en la clase que lanza el evento:

[GestorEventoArchivoDescargado](#)

```
public delegate void GestorEventoArchivoDescargado(object fuente, EventArgs args); //Delegado
```

Donde fuente será el objeto que lance el evento y args los argumentos que aportan información sobre el mismo.

- 2) Creamos el evento basado en el delegado:

```
public event GestorEventoArchivoDescargado ArchivoDescargado; //Evento
```

- 3) Creamos el método disparador del evento. Se llamará igual que el evento, pero delante le ponemos la preposición "En" u "On" en inglés.

En este caso: [EnArchivoDescargado](#).

```
//Paso 3: método que dispara el evento
0 referencias
protected virtual void EnarchivoDescargado()
{
    if (ArchivoDescargado != null) {
        ArchivoDescargado(this, EventArgs.Empty);
    }
}
```

En este método se comprueba si el evento está activo, si alguien ha recibido ese evento. Se podría haber simplificado:

[ArchivoDescargado?.Invoke\(this, EventArgs.Empty\)](#)

- 4) Completamos el método Descargar con la llamada al "disparador",

[EnArchivoDescargado\(\)](#):

```
//Método que simula la descarga esperando 4 segundos a través de un Thread.
1 referencia
public void Descargar(Archivo archivo)
{
    Console.WriteLine("Descargando el archivo...");
    Thread.Sleep(4000);
    //Llamamos al método que lanza el evento.
    EnarchivoDescargado();
}
```



Clase Program

- 1) Definimos la clase que recibe la activación del evento:

[ServicioDescomprimir](#)

```

0 referencias
public class ServicioDescomprimir
{
    // 0 referencias
    public void EnArchivoDescargado(object fuente, EventArgs e)
    {
        //Simulamos la descompresión mostrando este mensaje por pantalla
        Console.WriteLine("ServicioDescomprimir: descomprimiendo el archivo...");
    }
}

```

2) Creamos una instancia del receptor en la clase Program:

```

var archivo = new Archivo() { Titulo="Archivo 1"};
var asistenteDescarga = new AsistenteDescarga();//emisor
var servicioDescomprimir= new ServicioDescomprimir();//receptor

```

ServicioDescomprimir.

3) Vinculamos el método del receptor al evento (que es como una instancia del delegado) para que se ejecute cuando se produzca el evento. Como recordaremos, debía tener la misma firma que el delegado.

```

asistenteDescarga.ArchivoDescargado += servicioDescomprimir.EnArchivoDescargado;

```

4) Añadimos debajo un Console.ReadKey(); para que no se cierre el programa.

Ejecutamos el programa.

Debería salir lo siguiente por pantalla:

```

C:\WINDOWS\system32\cmd.  X  +  v

Descargando el archivo...
ServicioDescomprimir: descomprimiendo el archivo...

```

 Quitamos la clase ServicioDescomprimir del archivo .cs Program, pulsando el destornillador que sale a la izquierda y dándole a "MoverTipo a ServicioDescomprimir.cs."

→ ¿Cómo implementaríamos el segundo receptor: ServicioNotificacion?

Una vez hayamos hecho esto, utilizaremos el parámetro de tipo EventArgs para pasar información, para ello:

a) Creamos una clase específica para este parámetro dentro de la clase que lanza el evento: [ArchivoEventArgs](#) que hereda de EventArgs.

```

namespace EventosyDelegados
{
    0 referencias
    public class ArchivoEventArgs : EventArgs
    {
        0 referencias
        public Archivo archivo { get; set; }
    }
    1 referencia
    public class AsistenteDescarga
    {

```

NOTA: poner en mayúscula el nombre de la propiedad: public Archivo
 Archivo{get;set;}

- b) Modificamos en la declaración del delegado, el segundo argumento, cambiándolo por un objeto de la clase nueva que hemos creado:

```

public delegate void GestorEventoArchivoDescargar(object fuente, ArchivoEventArgs args);

```

- c) Modificamos el método Descarga para usar el archivo que se le pasa como parámetro:

```

public void Descarga(Archivo archivo)
{
    Console.WriteLine("Archivo descargado...");
    Thread.Sleep(4000); // Simulamos la descarga

    // Paso 3.1
    EnArchivoDescargado(archivo);
}

```

- d) Modificamos la firma del método EnArchivoDescargado para que pueda recoger ese parámetro que le pasamos desde Descarga y también el segundo parámetro de la llamada a ArchivoDescargado (evento/instancia de delegado), sustituyéndolo por una instancia de la nueva clase que hemos creado ArchivoEventArgs, que creamos en ese mismo espacio, pasándole como nombre para la propiedad el archivo que tenemos como entrada del método EnArchivoDescargado.

```

protected virtual void EnArchivoDescargado(Archivo archivo)
{
    // Se verifica si el evento es nulo; es decir, si "alguien" recibe el evento
    if (ArchivoDescargado != null)
    {
        ArchivoDescargado(this, new ArchivoEventArgs() { Archivo = archivo });
    }
    // Se podría haber puesto de forma simplificada
    // ArchivoDescargado?.Invoke(this, EventArgs.Empty);
}

```

- e) Hacemos uso del parámetro **archivo** en el receptor, para lo cual, cambiamos también su firma, haciéndola coincidir con la del delegado, y mostramos en el mensaje el título del objeto archivo que hemos recibido como parámetro en los argumentos:

```
1 referencia
public class ServicioDescomprimir
{
    1 referencia
    public void EnArchivoDescargado(Object fuente, ArchivoEventArgs e)
    {
        //Simulamos que se descarga
        Console.WriteLine("ServicioDescomprimir: Descomprimiendo el archivo"+ e.Archivo.Titulo);
    }
}
```

Obtendremos la siguiente salida:

```
C:\WINDOWS\system32\cmd. x + v
Archivo descargado...
ServicioDescomprimir: Descomprimiendo el archivo Archivo 1
ServicioNotificación: El archivo se ha descargado y descomprimido.
```

➡ NOTA: Podíamos no haber creado nuestro propio delegado y usar uno que ya nos propone C#: **EventHandler**.

Se harían los siguientes cambios en la clase AsistenteDescarga:

1. Comentamos la declaración del delegado y su instancia.
2. Directamente en la instancia, en lugar de poner el delegado que habíamos creado, ponemos **EventHandler<ArchivoEventArgs>** y como argumento la clase que hemos creado que heredaba de EventArgs.

```
public class AsistenteDescarga
{
    //Paso 1: Declaramos el delegado, que tiene como argumentos el objeto que dispara el evento y
    //el conjunto de argumentos con más información del evento si la hubiera
    //public delegate void GestorEventoArchivoDescargar(object fuente, ArchivoEventArgs args);

    //Paso 2: Crear el evento, basado en el delegado
    //public event GestorEventoArchivoDescargar ArchivoDescargado;
    public event EventHandler<ArchivoEventArgs> ArchivoDescargado;
}
```

4. WPF (WINDOWS PRESENTATION FOUNDATION)

4.1. ¿Qué es WPF?

Es una tecnología de Microsoft que permite desarrollar interfaces de usuario enriquecidas para aplicaciones de escritorio en el entorno de Windows. Se basa en XAML (eXtensible Application Markup Language) para diseñar la interfaz de usuario y en .NET para gestionar la lógica y el comportamiento de la aplicación.

CARACTERÍSTICAS

- ❖ **Separación de diseño y lógica:** WPF permite separar el diseño visual de la lógica del programa. El diseño se define en archivos XAML, mientras que el comportamiento se programa en C# o Visual Basic .NET.
- ❖ **Soporte para gráficos avanzados:** Ofrece una amplia gama de capacidades gráficas, como animaciones, 3D, estilos personalizados, y efectos visuales, permitiendo crear interfaces mucho más atractivas y dinámicas que en Windows Forms.
- ❖ **Data Binding (Vinculación de datos):** WPF facilita el trabajo con datos a través de la vinculación de datos. Esto permite conectar componentes visuales a datos en tiempo real, ideal para aplicaciones que deben mostrar información constantemente actualizada.
- ❖ **Plantillas y estilos:** Con WPF, puedes definir estilos y plantillas que personalizan el aspecto y el comportamiento de los controles, asegurando una apariencia uniforme en toda la aplicación.
- ❖ **Soporte para MVVM:** WPF se diseñó para trabajar bien con el patrón de diseño Model-View-ViewModel (MVVM), que separa los datos, la interfaz de usuario y la lógica de negocio.

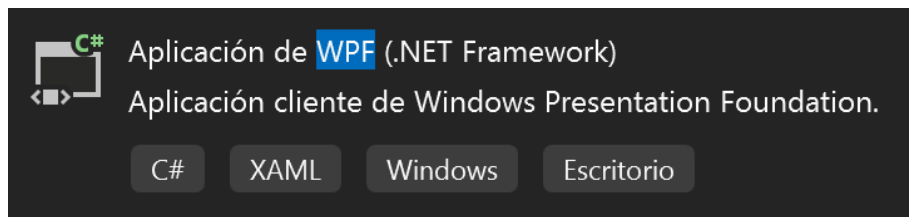
4.2. ¿Qué es XAML?

eXtensible Application Markup Language es un lenguaje de marcado basado en XML que se usa para definir la interfaz de usuario en aplicaciones WPF (Windows Presentation Foundation).

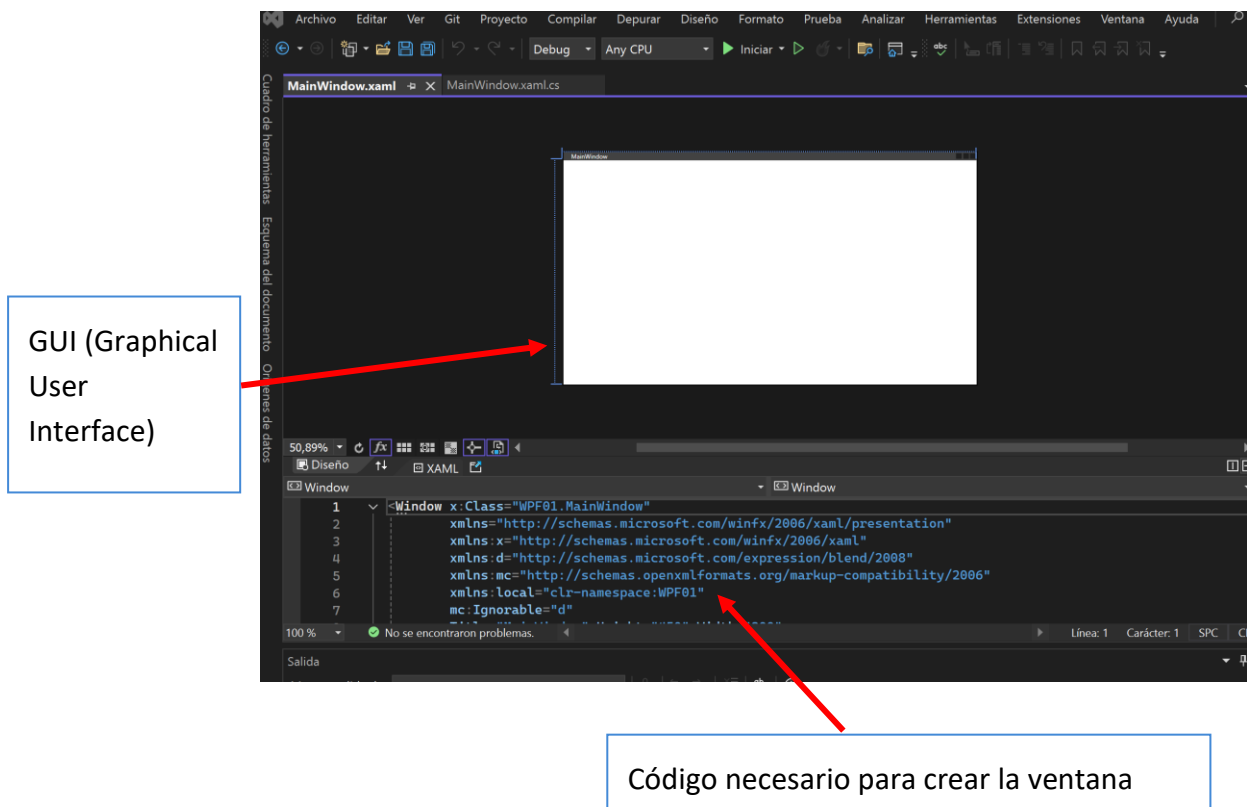
En WPF, **XAML permite describir de manera declarativa los elementos visuales de una aplicación**, como [ventanas](#), [controles](#), [estilos](#) y [animaciones](#), sin escribir código imperativo en C#. Esto significa que puedes definir la estructura, el diseño y la apariencia de la interfaz visual directamente en archivos XAML, facilitando la separación de la lógica de negocio (en C#) y el diseño de la UI.

4.3. Primer Proyecto: Hola mundo

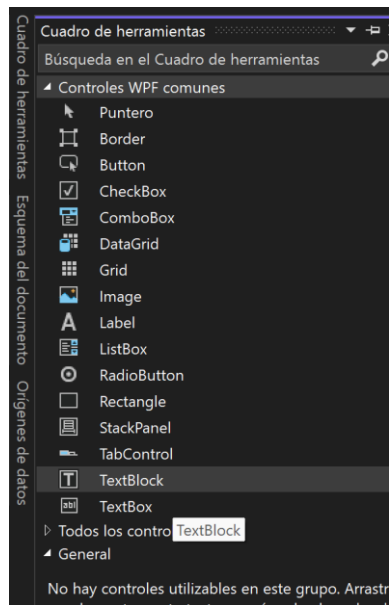
1. Creamos un proyecto de tipo:



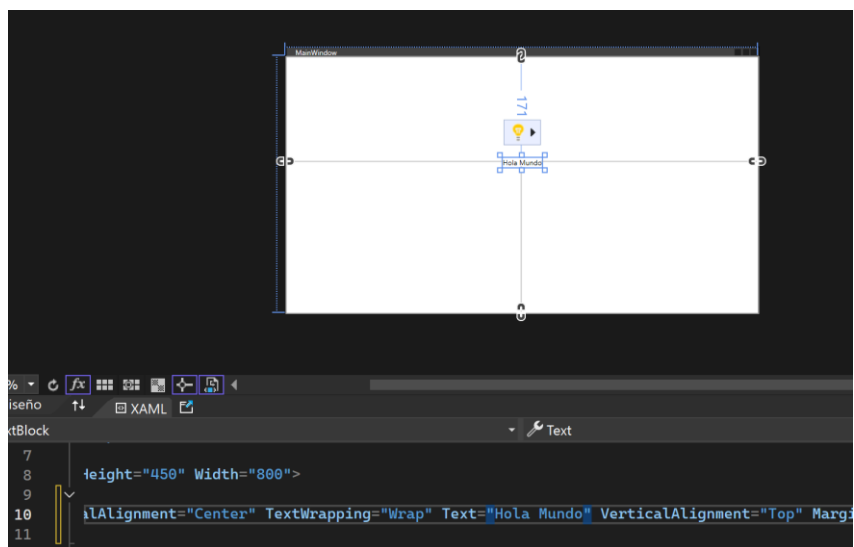
2. Aparecerá la siguiente ventana en un nuevo fichero .xaml



3. Abrimos el cuadro de herramientas e insertamos un *textblock*:



4. Escribimos “Hola Mundo” en la etiqueta de texto y ejecutamos:



COMENTARIOS

Para introducir un comentario en el código XAML lo haremos de la siguiente manera:

```
<!--XAML Extensive Application Markup Language-->
```

SINTAXIS

Ejemplo: creamos un botón

Si agregamos un elemento, como por ejemplo un botón, y le queremos poner propiedades, como el contenido, el alto, el ancho o el tamaño de letra, podríamos hacerlo de varias formas:

1. Añadiendo las propiedades dentro del primer <>.

```
<Grid>
  <Button Content="Click aquí" Height="50" Width="150" Click="Button_Click" FontSize="32"></Button>
</Grid>
```

NOTA: Click es el suceso que lanza el evento y ButtonClick es el método que está a la "escucha".

2. Añadiendo el contenido entre <Button>y</Button> :

```
<Grid>
  <Button Height="50" Width="150" Click="Button_Click" FontSize="32">Click Aquí</Button>
</Grid>
```

3. Poniendo otras propiedades entre las dos etiquetas Button de la siguiente forma:

```
<Grid>
  <Button Height="50" Width="150" Click="Button_Click">
    <Button.FontSize>32</Button.FontSize>
    <Button.Content>Click Aquí</Button.Content>
  </Button>
</Grid>
```

Otra forma de crear un botón es con el elemento `wrapPanel` que permite agregar muchos elementos con diferentes características dentro de un mismo botón.

```
<Grid>
  <Button Height="50" Width="300" Click="Button_Click">
    <Button.FontSize>32</Button.FontSize>
    <WrapPanel>
      <TextBlock Foreground="Blue">Botón</TextBlock>
      <TextBlock Foreground="Red">multi</TextBlock>
      <TextBlock Foreground="Green">color</TextBlock>
    </WrapPanel>
  </Button>
</Grid>
```

Quedaría este resultado:



Si quisiéramos hacer esto mismo escribiendo el código en C#, lo haríamos en el fichero .cs que se nos ha creado junto con el proyecto, poniendo el siguiente código:

```
public partial class MainWindow : Window
{
    0 referencias
    public MainWindow()
    {
        InitializeComponent();

        Grid grid = new Grid(); //Creamos la ventana
        this.Content = grid; //Al contenido de la clase MainWindow le asignamos el Grid creado
        Button btn= new Button(); //Creamos un botón
        btn.FontSize = 26; //Ponemos tamaño de letra 26

        WrapPanel wrapPanel=new WrapPanel(); //Creamos un WrapPanel
        TextBlock txt= new TextBlock(); //Creamos un bloque de texto
        txt.Text = "Botón"; //Escribimos botón en el bloque de texto
        txt.Foreground=Brushes.Blue; //Ponemos el texto en color azul
        wrapPanel.Children.Add(txt); //Agregamos el bloque de texto al WrapPanel

        txt= new TextBlock();
        txt.Text = "multi";
        txt.Foreground = Brushes.Red;
        wrapPanel.Children.Add(txt);

        txt = new TextBlock();
        txt.Text = "color";
        txt.Foreground = Brushes.Green;
        wrapPanel.Children.Add(txt);

        btn.Content=wrapPanel; //Agregamos al contenido del botón el WrapPanel creado
        grid.Children.Add(btn); //A la ventana le agregamos el botón con todo lo que hemos insertado dentro
    }
}
```

4.4. StackPanel, ListBox y Árboles Visual y Lógico

¿QUÉ ES UN STACKPANEL?

Un StackPanel es un contenedor que organiza sus elementos secundarios en una pila, ya sea en dirección vertical o horizontal. Es uno de los paneles de disposición (layout panels) que permite colocar y alinear los elementos de manera sencilla.

Ejemplo:

★ Creamos el StackPanel y añadimos un bloque de texto dentro y 3 ListBox, además de un botón, al que añadimos el evento click, que configuraremos en C#.

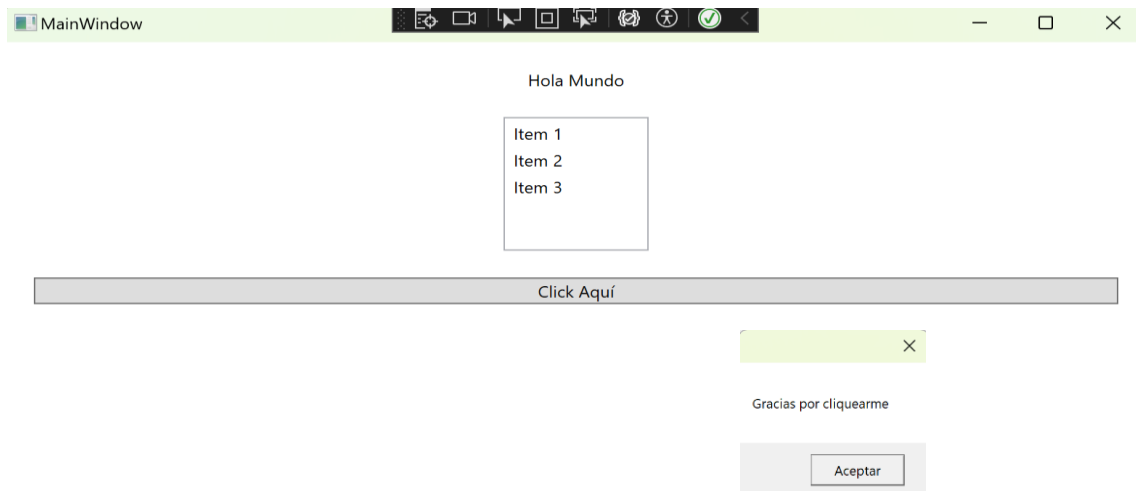
Un ListBox es un control que permite mostrar una lista de elementos al usuario, permitiendo también la selección de uno o más de esos elementos.

```
<Window x:Class="WPF01.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:WPF01"
        mc:Ignorable="d"
        Title="MainWindow" Height="450" Width="800">
    <StackPanel>
        <TextBlock HorizontalAlignment="Center" Margin="20">Hola Mundo</TextBlock>
        <ListBox Height="100" Width="100">
            <ListBoxItem Content="Item 1"></ListBoxItem>
            <ListBoxItem Content="Item 2"></ListBoxItem>
            <ListBoxItem Content="Item 3"></ListBoxItem>
        </ListBox>
        <Button Content="Click Aquí" Margin="20" Click="Button_Click"></Button>
    </StackPanel>
</Window>
```

En el código de C# aparece el método `Button_Click` al que le añadiremos una línea de código para que simplemente muestre un mensaje por pantalla al hacer click en el botón:

```
private void Button_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Gracias por hacer click");
}
```

Ejecutamos y vemos el resultado.

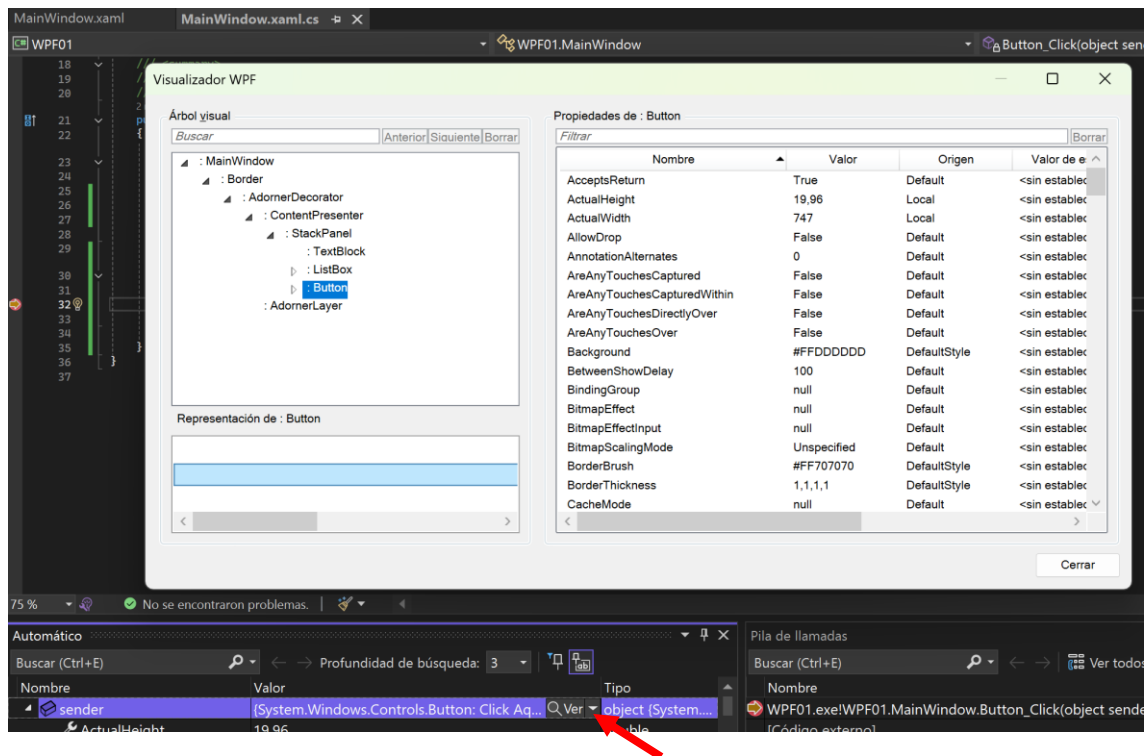


¿COMO ES LA LÓGICA DETRÁS DEL BOTÓN COMPARÁNDOLA CON LO VISTO ANTERIORMENTE?

Concepto en C#	Equivalente en WPF (Ej: Button Click)	Función / Rol
Delegado (delegate void MiDelegado(...)) o EventHandler	RoutedEventHandler	Tipo de método que puede manejar el evento Click. Define la firma que deben tener los métodos manejadores: (object sender, RoutedEventArgs e).
Evento (public event MiDelegado AlgoOcurre)	Button.Click → RoutedEventArgs	Representa la acción que puede ocurrir , en este caso el clic del botón. Internamente, se conecta con el RoutedEventArgs registrado .
Manejador / suscriptor (MiMetodo)	Método Boton_Click	Es el método que se ejecuta cuando ocurre el evento . Debe coincidir con el delegado (firma).
sender	sender del RoutedEventArgs	Referencia al control que disparó el evento , igual que en sus ejercicios: permite saber qué objeto generó la acción.
EventArgs / argumentos	RoutedEventArgs	Contiene información sobre el evento, en WPF además incluye información de la propagación (bubbling/tunneling).
Subscribir al evento (obj.AlgoOcurre += MiMetodo)	En XAML: Click="Boton_Click" (equivale a += new RoutedEventArgs(Boton_Click))	Conecta el evento con su manejador, para que se ejecute cuando ocurra el Click.

Aclaración: Click **no hereda de RoutedEventArgs**; es un evento de C# del tipo **RoutedEventArgs**. Pero internamente, WPF **registra Click como un RoutedEventArgs** para manejar la propagación por la jerarquía de controles.

★ Si ahora ponemos un punto de interrupción en la línea de código de este mensaje, abriremos el modo depuración y podremos ver el Árbol visual de los elementos que estamos creando:



Esto nos permite ir al detalle de la estructura que estamos creando.

4.5. Eventos RoutedEvents_3 Tipos: Direct, Bubbling y Tunneling

¿QUÉ SON ROUTEDEVENTS?

Son eventos que son enviados desde un objeto hacia otra parte del programa. Es el "motor interno" que define cómo se propaga el evento en la jerarquía

A diferencia de los eventos típicos, que solo se manejan en el lugar donde se originan, los Routed Events pueden propagarse a otros elementos, lo que permite que los eventos desencadenados por una acción (como un clic o una pulsación de tecla) sean capturados y manejados por diferentes niveles en la jerarquía de la interfaz de usuario (árbol visual).

Hay tres tipos:

1. DirectEvent

Estos eventos solo se manejan en el mismo elemento donde se originan, sin propagarse hacia otros elementos. El botón creado anteriormente sería de este tipo.

Ejemplo: click

2. BubblingEvent

Estos eventos comienzan en el elemento que los dispara y "burbujean" hacia arriba en la jerarquía de elementos visuales hasta llegar al elemento raíz (como la ventana).

Envía el evento al tope por arriba de la jerarquía del árbol visual, hasta que alguno de los elementos que hay por encima, maneja el evento desencadenado.

Ejemplo: MouseUp (se hace algo cuando se hace click, se mantiene y después se suelta)

Se utilizan cuando necesitas que el evento sea manejado por un contenedor o algún ancestro del elemento que lo generó.

3. TunnelingEvent

Estos eventos empiezan en el elemento raíz y "túnelan" en el mismo elemento o hacia abajo hasta llegar al elemento que lo dispara, dentro del árbol visual.

En WPF, los eventos de túnel suelen llevar el prefijo "Preview" (por ejemplo, PreviewMouseDown).

Son útiles cuando necesitas manejar un evento antes de que llegue al elemento específico, aplicando alguna lógica o condición previa.

Ejemplo: PreviewMouseUp

Ejemplo 1: Evento Bubbling

El más común.

Ocurre cuando, por ejemplo, haces clic en un Button, pero el evento también puede ser capturado por su contenedor (StackPanel, Grid, Window, etc.).

XAML

```
<Window x:Class="Ejemplo_Grid.MainWindow"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  xmlns:local="clr-namespace:Ejemplo_Grid"
  mc:Ignorable="d"
  Title="MainWindow" Height="200" Width="300">
  <StackPanel Background="■ LightBlue" Button.Click="StackPanel_Click">
    <Button Content="Haz clic aquí" Click="Button_Click" Margin="10"/>
  </StackPanel>
</Window>
```

C# Code behind

```

1 referencia
private void Button_Click(object sender, RoutedEventArgs e)
{
    ...
    MessageBox.Show("El botón recibió el clic (Bubbling empieza aquí)");
}

1 referencia
private void StackPanel_Click(object sender, RoutedEventArgs e)
{
    ...
    MessageBox.Show("El StackPanel también recibió el clic (el evento subió)");
}

```

Y ejecutamos. **Qué ocurre:**

1. Haces clic en el botón.
2. Se lanza Button.Click.
3. El evento “burbujea” hacia arriba y llega al StackPanel.

Aclaración: El evento que se está “escuchando” es **Button.Click**, no un evento del StackPanel. Lo que hace WPF aquí es decirle al StackPanel:

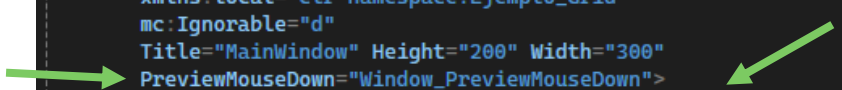
“Si dentro de ti ocurre un evento Click que viene de un Button, llámame al método StackPanel_Click.” Por eso no necesitas (ni puedes) escribir Click="StackPanel_Click" en el StackPanel, porque **StackPanel no tiene un evento Click propio**.

El evento **Button.Click="StackPanel_Click"** está definido **dentro del StackPanel**, pero no como StackPanel.Click.

Ejemplo 2: Evento Tunneling

Este ocurre antes que el *Bubbling*, y en sentido contrario: **de arriba hacia abajo**. Todos los eventos de túnel empiezan con “**Preview**”.

XAML



```

<Window x:Class="Ejemplo_Grid.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:Ejemplo_Grid"
        mc:Ignorable="d"
        Title="MainWindow" Height="200" Width="300"
        PreviewMouseDown="Window_PreviewMouseDown">
    <StackPanel PreviewMouseDown="StackPanel_PreviewMouseDown" Background="■"LightCoral">
        <Button Content="Haz clic aquí" PreviewMouseDown="Button_PreviewMouseDown" Margin="10"/>
    </StackPanel>
</Window>

```

C# Code behind

```

private void Window_PreviewMouseDown(object sender, MouseButtonEventArgs e)
{
    MessageBox.Show("El evento empezó en la Window (Tunneling)");
}

1 referencia
private void StackPanel_PreviewMouseDown(object sender, MouseButtonEventArgs e)
{
    MessageBox.Show("Luego pasó por el StackPanel (Tunneling)");
}

1 referencia
private void Button_PreviewMouseDown(object sender, MouseButtonEventArgs e)
{
    MessageBox.Show("Y finalmente llegó al Button (Tunneling termina aquí)");
}

```

Cuando el evento transporta **información adicional** (como coordenadas del ratón o teclas pulsadas), WPF no usa directamente RoutedEventArgs, sino **una subclase especializada**.

Por ejemplo:

Evento	Clase de argumentos	Propiedades adicionales
MouseDown, PreviewMouseDown	MouseButtonEventArgs	Qué botón se pulsó, posición del ratón, clics, etc.
KeyDown, PreviewKeyDown	KeyEventArgs	Qué tecla se pulsó, modificadores (Ctrl, Alt, etc.)
TextInput	TextCompositionEventArgs	Qué texto se escribió
Click	RoutedEventArgs	(no necesita datos extra)

Ejemplo 3: Evento **Direct**

No viaja ni sube ni baja. Solo afecta al elemento donde ocurre.

XAML

```

<StackPanel Background="GreenYellow">
    <Button Content="Pasa el ratón por aquí" MouseEnter="Button_MouseEnter" Margin="10"/>
</StackPanel>

```

C# Code behind

```
private void Button_MouseEnter(object sender, MouseEventArgs e)
{
    MessageBox.Show("El evento MouseEnter es directo: solo el botón lo recibe.");
}
```

¿Qué tipo de Routed Event tiene el evento click?

4.6. Grid

El Grid es un contenedor de diseño que organiza elementos en una cuadrícula de filas y columnas.

Ejemplo 1.

1. Creamos dos columnas a las que les agregamos el texto “Hola”

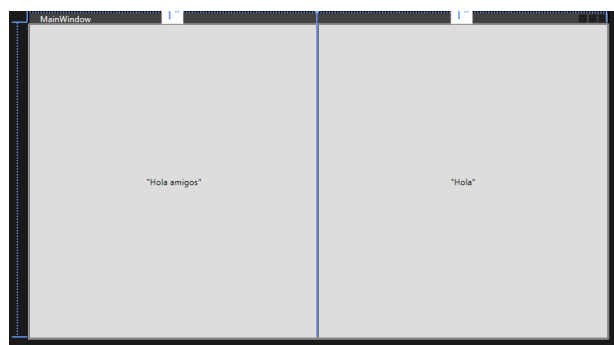
```
<Grid>
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="100"/>
    <ColumnDefinition Width="100"/>
  </Grid.ColumnDefinitions>
  <Button Grid.Column="0">Hola</Button>
  <Button Grid.Column="1">Hola</Button>
</Grid>
```

2. Si quisiéramos que el ancho se adaptase al tamaño del texto, pondríamos la palabra “auto”.

```
<ColumnDefinition Width="auto"/>
```

3. Si le ponemos un “*” en el ancho, le estamos diciendo que ocupe todo el espacio que tenga disponible.

```
<ColumnDefinition Width="*" />
<ColumnDefinition Width="*" />
```



Según el número de asteriscos le reservará una porción mayor de la ventana a la columna.

4. Si quisiéramos ponerle dos filas a cada columna y mover el “Hola” a la fila de abajo añadiríamos:

```
<Grid.RowDefinitions>
```


En la siguiente forma:

```
<Grid>
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="3*" />
    <ColumnDefinition Width="*" />
  </Grid.ColumnDefinitions>
  <Grid.RowDefinitions>
    <RowDefinition Height="*" />
    <RowDefinition Height="*" />
  </Grid.RowDefinitions>
  <Button Grid.Column="0">"Hola amigos"</Button>
  <Button Grid.Column="1" Grid.Row="1">"Hola"</Button>
</Grid>
```

a. Manejo de excepciones

Siempre que tienes una sección de código que es muy probable que arroje una excepción, entonces deberías encapsularla en un bloque de try-catch para manejar la excepción satisfactoriamente. Considera el siguiente ejemplo:

1. Creamos un botón sencillo con un evento de tipo "click":

```
<Grid>
  <Button Content="Haz click aquí" Height="60" Width="150" Click="Button_Click"></Button>
</Grid>
```

2. Completamos el método que maneja el evento de la siguiente forma:

```
private void Button_Click(object sender, RoutedEventArgs e)

{

    string s = null;

    s.Trim();

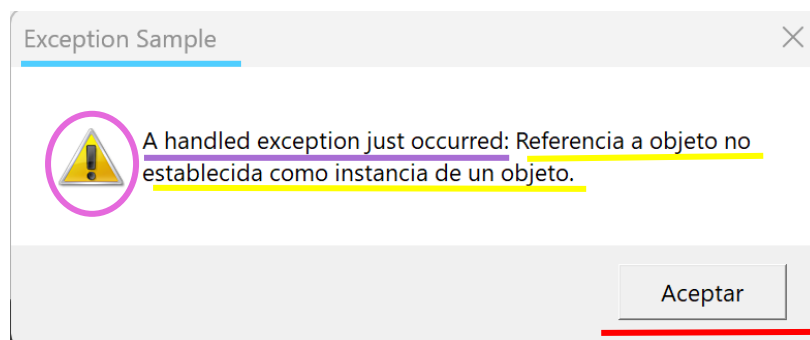
}
```

3. Ejecutamos y, obviamente, nos salta una excepción.

Lo adecuado, por tanto, será encerrar nuestro código en un bloque Try catch como el siguiente:

```
try
{
    s.Trim();
}
catch (Exception ex)
{
    MessageBox.Show("A handled exception just occurred: " + ex.Message,
        "Exception Sample", MessageBoxButton.OK, MessageBoxImage.Warning);
}
```

Al ejecutarlo de nuevo, nos mostrará lo siguiente:



5. CONTROLES BÁSICOS

5.1. TEXTBLOCK

Como el control **TextBlock** no hereda de la clase Control, no debería considerarse un control, pero como su uso en el framework WPF es similar al de cualquier otro control y por razones de simplicidad lo trataremos como un control.

Aunque el control **TextBlock** es uno de los más básicos de WPF, es de mucha utilidad. Te permite colocar texto en la pantalla, de manera similar a como lo hace el control Label, pero más simple y consumiendo menos recursos.

Normalmente se usa Label para textos cortos de una línea (que pueden contener, por ejemplo, una imagen). Por otra parte, TextBlock trabaja muy bien con cadenas de texto de varias líneas, pero solo puede contener texto(strings). Tanto el Label como el TextBlock ofrecen cada uno ventajas únicas, por lo que su uso depende mucho de la situación.

MANEJO DE TEXTOS LARGOS

[Ejemplo_Introducir textos más largos con TextBlock: opciones](#)

```

<Window x:Class="WpfTutorialSamples.Basic_controls.TextBlockSample"

    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

    Title="TextBlockSample" Height="100" Width="200">

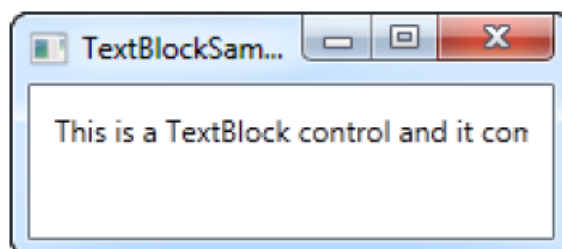
    <Grid>

        <TextBlock Margin="10">This is a TextBlock control and it comes with a very
long text</TextBlock>

    </Grid>

</Window>

```



Como apreciarás enseguida de la captura de pantalla, el TextBlock es perfectamente capaz de lidiar con textos largos y de múltiples líneas, pero no hace nada por defecto. En este caso el texto es demasiado largo para ser mostrado dentro de la ventana, entonces WPF muestra todo el texto que puede y luego simplemente lo corta.

Vamos a ver 3 maneras de solucionar esto:

```
<StackPanel>
```

```
<TextBlock Margin="10" Foreground="Red">
```

This is a TextBlock control<LineBreak />

with multiple lines of text.

```
</TextBlock>
```

```
<TextBlock Margin="10" TextTrimming="CharacterEllipsis"
```

```
Foreground="Green">
```

This is a TextBlock control with text that may not be rendered completely, which will be indicated with an ellipsis.

```
</TextBlock>
```

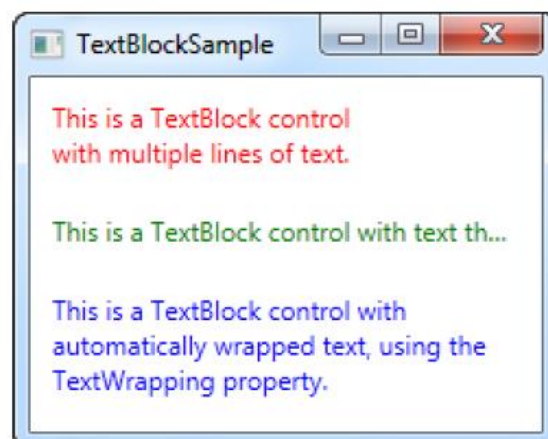
```
<TextBlock Margin="10" TextWrapping="Wrap" Foreground="Blue">
```

This is a TextBlock control with automatically wrapped text, using the TextWrapping property.

```
</TextBlock>
```

```
</StackPanel>
```

```
</Window>
```



Ahora, tenemos tres controles `TextBlock`, cada uno con un color diferente (usando la propiedad `Foreground`) para una mejor visualización. Todos ellos controlan el hecho de que su contenido de texto es demasiado largo de formas diferentes:

El `TextBlock` rojo usa una etiqueta **`LineBreak`** para manualmente dividir la línea en una determinada posición. Esto da absoluto control sobre dónde quieres dividir el texto en una nueva línea, pero no es muy flexible para la mayoría de situaciones. Si el usuario hace más grande la ventana, el texto se ajustará en la misma posición, aunque ahora puede haber espacio suficiente para ajustar el texto entero en una línea.

El `TextBlock` verde usa la propiedad **`TextTrimming`** con el valor **`CharacterEllipsis`** para hacer que el `TextBlock` muestre elipsis (...) al final cuando no puede ajustar más texto dentro del control. Esto es una manera común de mostrar que hay más texto, pero no suficiente sitio para mostrarlo. Esto es genial cuando tienes texto que podría ser demasiado largo pero tú no quieres usar más de una línea. Como alternativa a **`CharacterEllipsis`** tú puedes usar **`WordEllipsis`**, el cual corta el texto al final de la última palabra posible en vez del último carácter posible, previniendo que una sola palabra sea mostrada solo en parte.

El `TextBlock` azul usa la propiedad **`TextWrapping`** con el valor **`Wrap`**, para hacer que el texto del `TextBlock` salte a la siguiente línea cuando no pueda ajustar más texto dentro de la línea anterior. Contrario al primer `TextBlock`, donde nosotros manualmente definimos donde salta el texto, esto sucede automáticamente e incluso mejor: Es automáticamente ajustado tan pronto como el `TextBlock` obtenga más o se quede con menos espacio disponible.

BOLD. ITALIC Y UNDERLINE

Estos son, probablemente, los elementos en línea más simples. Sus nombres en inglés dicen mucho sobre lo que hacen, pero aun así veamos cómo utilizarlos con un rápido ejemplo:

```

<Window x:Class="WpfTutorialSamples.Basic_controls.TextBlockInlineSample"

    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"

    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"

    Title="TextBlockInlineSample" Height="100" Width="300">

    <Grid>

        <TextBlock Margin="10" TextWrapping="Wrap">

            TextBlock with <Bold>bold</Bold>, <Italic>italic</Italic> and

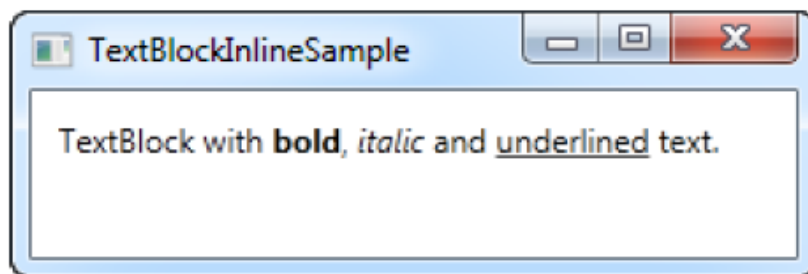
            <Underline>underlined</Underline> text.

        </TextBlock>

    </Grid>

```

Obtendremos lo siguiente a la salida:



De forma similar al HTML, simplemente encierra el texto deseado con una etiqueta Bold para obtener texto en negrita, y así con el resto de etiquetas. Eso hace que sea muy sencillo crear y mostrar diversos textos en tus aplicaciones.

Estas tres etiquetas son especializaciones del elemento Span, cada una estableciendo una propiedad concreta del elemento Span para obtener el efecto deseado. Por ejemplo, la etiqueta Bold simplemente establece la propiedad FontWeight del elemento Span subyacente, la etiqueta Italic establece la propiedad FontStyle y así sucesivamente.

LINEBREAK

Introduce un salto de línea forzado en un punto del texto. Vea el capítulo anterior para obtener un ejemplo de dónde usar el elemento LineBreak.

HYPERLINK

El elemento Hyperlink le permite introducir enlaces en sus textos, con un aspecto visual acorde con el tema de Windows activo, lo que suele ser alguna tonalidad de color azul cambiando a rojo cuando el cursor se coloca encima y teniendo éste último forma de mano. Puede usar la propiedad NavigateUri para definir la URL a la que desea navegar.

```
<Window x:Class="WpfTutorialSamples.Basic_controls.TextBlockHyperlinkSample"
```

```
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
```

```
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
```

```
Title="TextBlockHyperlinkSample" Height="100" Width="300">
```

```
<Grid>
```

```
<TextBlock Margin="10" TextWrapping="Wrap">
```

```
    This text has a <Hyperlink
```

```
    RequestNavigate="Hyperlink_RequestNavigate" NavigateUri="https://www.google.com">link</Hyperlink>  
    in it.
```

```
</TextBlock>
```

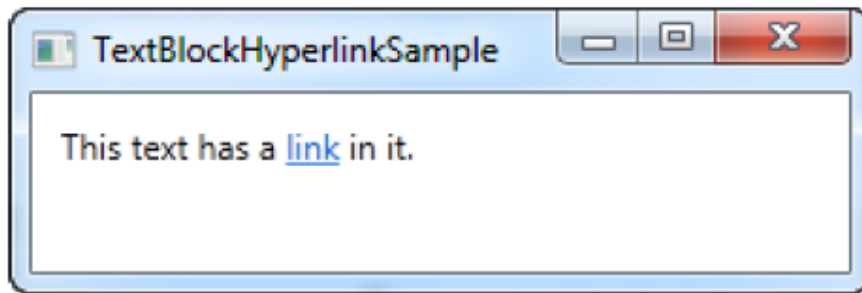
```
</Grid>
```

```
</Window>
```

NOTAS SOBRE EL CÓDIGO:

- **TextWrapping="Wrap"**: Permite que el texto se ajuste automáticamente dentro del área del TextBlock, haciendo que el texto se divida en varias líneas si es necesario.
- **RequestNavigate="Hyperlink_RequestNavigate"**: Define un controlador de eventos (Hyperlink_RequestNavigate) que se ejecuta cuando se hace clic en el hipervínculo.
- **NavigateUri="https://www.google.com"**: Especifica la URL a la que el hipervínculo dirigirá al usuario cuando haga clic en él. En este caso, la URL es "<https://www.google.com>".

Obtendríamos lo siguiente:



El Hyperlink también puede ser usado en páginas WPF, donde permite navegar entre diferentes páginas. En ese caso, no será necesario gestionar explícitamente el evento RequestNavigate, como hacemos en el ejemplo, pero para poder abrir URLs externas desde una aplicación de escritorio WPF, es necesario implementar este evento y hacer uso de la clase Process. Nos suscribimos al evento RequestNavigate, lo que nos permite abrir la URL enlazada en el navegador predefinido del usuario con un simple manejador de evento como este en el archivo de código subyacente:

```
private void Hyperlink_RequestNavigate(object sender, RequestNavigateEventArgs e)
{
    Process.Start(e.Uri.AbsoluteUri);
    e.Handled = true;
}
```

NOTAS SOBRE EL CÓDIGO:

- Process.Start es un método de la clase Process que se usa para iniciar un proceso nuevo. En este contexto, se utiliza para abrir la URL especificada en el navegador predeterminado del sistema operativo.
- **e.Handled = true;**: Esta propiedad Handled se encuentra en el objeto RequestNavigateEventArgs (representado por e). Al establecer e.Handled = true, indicas que el evento ya fue gestionado por tu código y que no es necesario que el sistema intente manejarlo de nuevo. Esto evita que otros controladores del mismo evento intenten hacer algo adicional con la navegación.
- En **.NET Framework**, Process.Start(e.Uri.AbsoluteUri); abre el enlace sin necesidad de ProcessStartInfo o UseShellExecute. En **.NET Core y versiones posteriores (.NET 5+ y .NET 6+)**, el comportamiento predeterminado de Process.Start cambió para mejorar la seguridad y el control sobre los procesos. Habríamos de poner:

```
Process.Start(new ProcessStartInfo(e.Uri.AbsoluteUri) { UseShellExecute = true })
```


RUN

<Run> es útil para dar formato a fragmentos de texto en TextBlock o dentro de un contenedor de texto de flujo. Permite una estructura y estilo flexibles sin tener que dividir el texto en varios elementos TextBlock.

Ejemplo sin run:

```
<TextBlock FontWeight="Bold" Foreground="Red" FontStyle="Italic">
    Este texto aplicará todos los estilos a cada palabra.
</TextBlock>
```

En este caso, todo el texto estará en negrita, en color rojo y en cursiva.

Ejemplo con run:

```
<TextBlock>
    Este texto tiene una palabra en <Run FontWeight="Bold">negrita</Run>,
    otra en <Run Foreground="Red">rojo</Run> y otra en <Run FontStyle="Italic">cursiva
</TextBlock>
```

Aquí puedes ver cómo, gracias a <Run>, solo ciertas palabras reciben estilos específicos.

SPAN

A diferencia de <Run> que solo puede contener texto. No puedes anidar otros elementos dentro de un <Run>, sin embargo **** es un contenedor en línea más flexible que puede agrupar **varios elementos en línea**, como <Run>, otros , e incluso <Hyperlink>. Esto te permite aplicar estilos o comportamientos a grupos de elementos de texto en lugar de a palabras o fragmentos individuales.

Ejemplo:

```

<TextBlock>
    Este es un texto normal con un <Span Foreground="Green">
        <Run FontWeight="Bold">texto en negrita</Run>
    y aquí tienes un
        <Hyperlink NavigateUri="https://www.example.com" Foreground="Blue">
            <Run Text="enlace" FontStyle="Italic"/>
        </Hyperlink>.
    </Span>
</TextBlock>

```

Quedaría de la siguiente forma:



5.2. El control LABEL

El control **Label**, en su forma más simple, se parecerá mucho al TextBlock que usamos en otro apartado. Sin embargo, rápidamente verás que en lugar de una propiedad de Text, tiene una propiedad **Content**. La razón de esto es que Label **puede alojar cualquier otro tipo de control directamente dentro de él, en lugar de solo texto**.

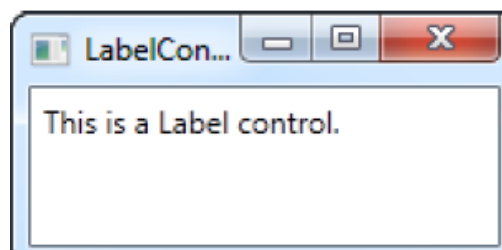
Este contenido también puede ser una cadena, como verás en este primer y muy básico ejemplo:

```

<Grid>
    <Label Content="This is a Label control." />
</Grid>

```

Obtendremos esta salida:



Otra cosa que puedes ver es que el control Label, por defecto, tiene un poco de padding (espacio vacío entre el contenido de un elemento y sus bordes internos), permitiendo al

texto ser renderizado a pocos pixels del alto y de la esquina izquierda. Este no es el caso para el TextBlock, donde tú puedes especificarlo manualmente.

En un caso sencillo como este, donde el contenido es simplemente una cadena de texto, el Label creará un TextBlock internamente y mostrará la cadena en él.

El control Label vs el control TextBlock

Así que ¿por qué usar el control Label? Bien, hay pocas diferencias importantes entre el Label y el TextBlock. El TextBlock solo permite renderizar una cadena de texto, mientras que el Label te permite:

- Especificar un borde
- Renderizar otros controles, por ejemplo, una imagen.
- Usar templates a través de la propiedad ContentTemplate
- **Usar teclas de acceso para posicionar el foco en el control relacionado**

El último punto es uno de los principales motivos para usar el Label en vez del control TextBlock. Cuando solo quieras renderizar texto sencillo, deberías usar el TextBlock, ya que es más ligero y funciona mejor que el Label en la mayoría de los casos.

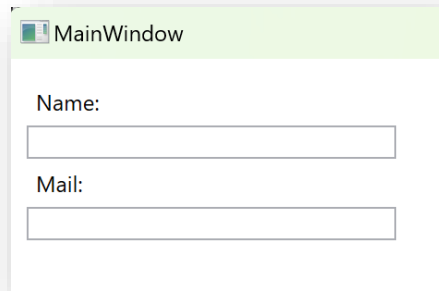
Label y Teclas de Acceso (mnemotécnicos)

En Windows y otros sistemas operativos también, es una práctica común que tú puedas acceder a controles en un diálogo pulsando la tecla [Alt] y pulsando el carácter que corresponda al control que deseas acceder. El carácter a presionar será sobresaltado cuando tú pulses la tecla [Alt]. El control TextBlock no soporta esta funcionalidad, pero el Label sí, así que **para controlar etiquetas, el control Label es normalmente una excelente opción.**

Veamos un ejemplo de ello en acción:

```
<StackPanel Margin="10">
  <Label Content="_Name:" Target="{Binding ElementName=txtName}" />
  <TextBox Name="txtName" Width="200" HorizontalAlignment="Left"/>
  <Label Content="_Mail:" Target="{Binding ElementName=txtMail}" />
  <TextBox Name="txtMail" Width="200" HorizontalAlignment="Left"/>
</StackPanel>
```

Obtendríamos la siguiente salida:



Intenta ejecutarlo, manteniendo presionada la tecla [Alt] y entonces presiona N y M. Verás como cambia el foco entre los dos Textbox.

NOTA DEL CÓDIGO:

- **Target="{Binding ElementName=txtName}"**: Este es el atributo clave que asocia el Label con otro control. En este caso, la propiedad **Target**, usando la propiedad **ElementName**, establece que el Label está **vinculado, usando enlace estándar de WPF (Bind)**, al TextBox llamado txtName, en el primer caso y txtMail en el segundo.
- Definimos la **tecla de acceso poniendo una barra baja (_) antes del carácter**. No tiene por qué ser el primer carácter, puede estar antes de cualquiera de los caracteres en el contenido de tu Label. Lo normal es usarlo en el primer carácter que no está siendo usado como tecla de acceso en otro control.

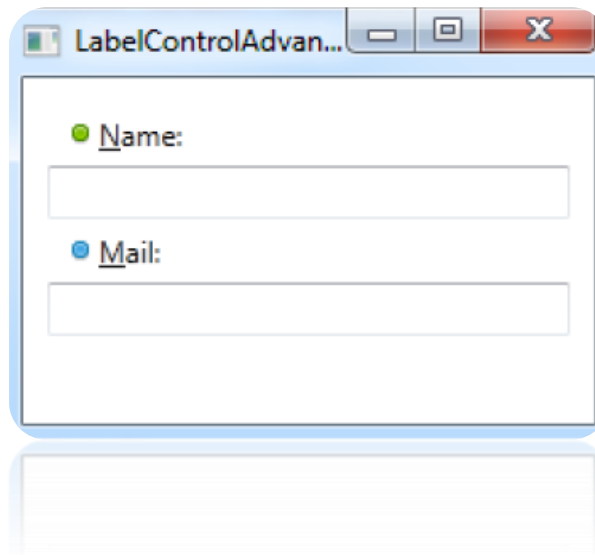
Usando controles como contenido de un LABEL

Como ya hemos comentado, el control Label te permite alojar otros controles, mientras mantiene los otros beneficios. Probemos con un ejemplo donde tenemos tanto una imagen como un trozo de texto dentro de un Label, mientras tenemos una tecla de acceso para cada uno de los Label:

```
<StackPanel Margin="10">
  <Label Target="{Binding ElementName=txtName}">
    <StackPanel Orientation="Horizontal">
      <Image Source="http://cdn1.iconfinder.com/data/icons/fatcow/16/bullet_green.png" />
      <AccessText Text="_Name:" />
    </StackPanel>
  </Label>
  <TextBox Name="txtName" Width="300" HorizontalAlignment="Left"/>
  <Label Target="{Binding ElementName=txtMail}">
    <StackPanel Orientation="Horizontal">
      <Image Source="http://cdn1.iconfinder.com/data/icons/fatcow/16/bullet_blue.png" />
      <AccessText Text="_Mail:" />
    </StackPanel>
  </Label>
  <TextBox Name="txtMail" Width="300" HorizontalAlignment="Left"/>
</StackPanel>
```

En vez de usar una simple cadena de caracteres, nuestro Label contiene tanto una imagen como un trozo de texto (dentro de un control AccessText, lo que nos permite usar la tecla de acceso para el Label). Ambos controles están dentro de un StackPanel horizontal, ya que el Label, como cualquier otro derivado de ContentControl, sólo puede contener un control hijo.

El control Image, descrito posteriormente en este tutorial, usa una imagen remota - esto es ÚNICAMENTE como demostración y NO es una buena idea en la mayoría de aplicaciones reales.



5.3. El control TEXTBOX

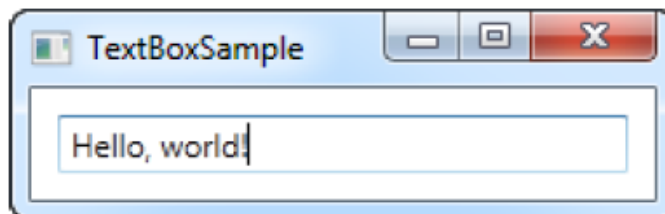
El control TextBox es el control más básico para introducir texto en WPF, permitiendo al usuario final escribir texto plano en una sola línea para un formulario o como múltiples líneas como en un editor de texto.

Anteriormente habíamos visto el control `TextBlock`. Señalar la diferencia de que **TextBlock** se utiliza principalmente para **mostrar texto** estático o de solo lectura en la interfaz de usuario. Es ideal cuando solo necesitas presentar información al usuario. Sin embargo, **TextBox** está diseñado para permitir la **entrada de texto por parte del usuario**. Los usuarios pueden escribir, editar y modificar texto en este control.

Probemos con un ejemplo simple:

```
<Grid>
  <StackPanel Margin="10" Height="30" Width="100">
    <TextBox Text="Hello, world"/>
  </StackPanel>
</Grid>
```

Ejecútalo y verás que puedes escribir en la “caja” de texto.



Intenta hacer clic derecho en el `TextBox`. Te aparecerá un menú con opciones permitiendo usar el `TextBox` con el Portapapeles de Windows. Las teclas de acceso directo por defecto para deshacer y rehacer (`Ctrl+Z` y `Ctrl+Y`) también deben funcionar.

TEXTBOX MULTILÍNEA

En el ejemplo anterior si agregas más texto que el que cabe en una sola línea, el control se desplaza. Sin embargo, convertir el `TextBox` a **multilínea** es muy sencillo.

```
<Grid Margin="10">
  <TextBox AcceptsReturn="True" TextWrapping="Wrap" />
</Grid>
```

Hemos agregado dos propiedades: la propiedad **AcceptsReturn** convierte el `TextBox` en un control multi-línea permitiendo el uso de la tecla `Enter/Return` para ir a la siguiente línea. La propiedad **TextWrapping** hace el texto continúe en la siguiente línea una vez que alcanza el final de una línea.

REVISIÓN ORTOGRÁFICA CON TEXTBOX

El control TextBox provee revisión automática de ortografía para Inglés y un par de otros lenguajes (al momento de escribir este documento, Inglés, Francés, Alemán y Español son lenguajes soportados).

Ejemplo:

```
<Grid Margin="10">  
    <TextBox AcceptsReturn="True" TextWrapping="Wrap" SpellCheck.IsEnabled="True" Language="es-ES" />  
</Grid>
```



NOTA: Solo funcionará con los paquetes de idiomas que tengamos instalados en el entorno. En inglés sería: "en-US"

SELECCIONES EN TEXTBOX

El TextBox permite la selección de texto; por ejemplo, para borrar una palabra completa o para copiar texto al portapapeles. El control TextBox de WPF posee ciertas propiedades para trabajar con el texto seleccionado, todas ellas son modificables.

En el siguiente ejemplo veremos estas propiedades:

```
<DockPanel Margin="10">  
    <TextBox Name="txtBox2DAM" SelectionChanged="txtBox2DAM_SelectionChanged" DockPanel.Dock="Top" />  
    <TextBox Name="txtStatus" AcceptsReturn="True" TextWrapping="Wrap" IsReadOnly="True" />  
</DockPanel>
```

El ejemplo consiste de dos controles TextBox: uno para edición y otro para salida del texto seleccionado. Para esto, indicamos la propiedad IsReadOnly en true, para prevenir la edición del TextBox. Nos subscribimos al evento SelectionChanged en el primer TextBox que se maneja en el code-behind:

```
1 referencia  
private void txtBox2DAM_SelectionChanged(object sender, RoutedEventArgs e)  
{  
    //TextBox textBox = sender as TextBox;  
    txtStatus.Text = "Selection starts at character #" + txtBox2DAM.SelectionStart +  
        Environment.NewLine;  
    txtStatus.Text += "Selection is " + txtBox2DAM.SelectionLength +  
        " character(s) long" + Environment.NewLine;  
    txtStatus.Text += "Selected text: '" + txtBox2DAM.SelectedText + "'";  
}
```

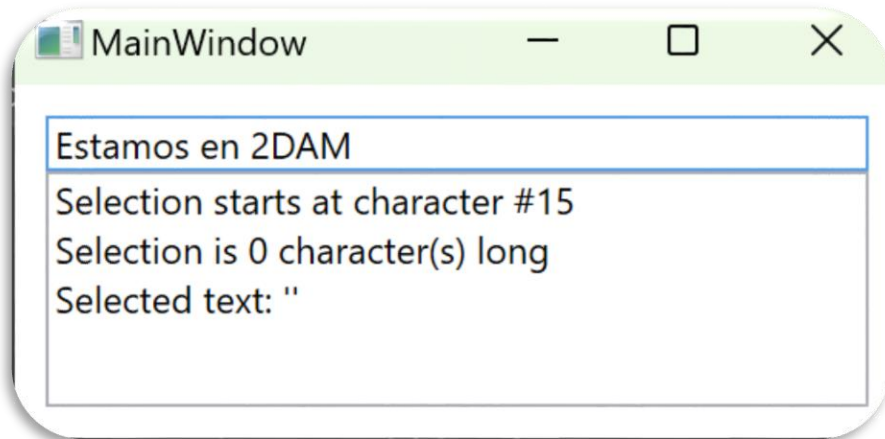
NOTA: el evento se lanza al hacer click sobre la zona de escritura del TextBox.

Usamos tres propiedades interesantes para lograr esto:

- **SelectionStart**, la cual nos brinda la posición actual del cursor o, si hay texto seleccionado, donde éste inicia.

- **SelectionLength**, nos brinda la longitud del texto seleccionado si hubiere. Si no, solo retorna 0.
- **SelectedText**, nos retorna el texto seleccionado si lo hubiere. Si no, se retorna un texto vacío.

Obtenemos la siguiente salida:



¿Qué es DockPanel y cómo funciona?

5.4. El control BUTTON

Como ya hemos adelantado, la creación de un botón en XAML pasa por usar la etiqueta `<Button>` y asignar el texto y las características que consideremos.

El ejemplo más simple sería el que sigue:



Si además quisiéramos que hiciera “algo” al ser pulsado, también hemos visto que se puede vincular un evento de distinta naturaleza al botón, tras lo cual, deberemos programar el método manejador de dicho evento en C#:

XAML

```
<Button Click="HelloWorldButton_Click">Hello, World!</Button>
```

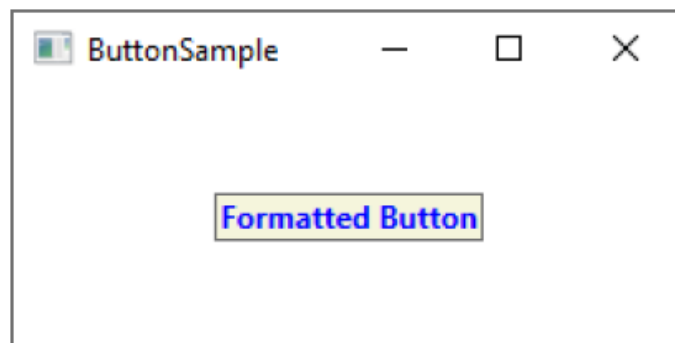
C#


```
private void HelloWorldButton_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Hello, world!");
}
```

FORMATEO DEL CONTENIDO

Internamente, el texto simple dentro del Contenido del botón se convierte en un control TextBlock, lo que también significa que puedes controlar los mismos aspectos del formato del texto. Encontrarás varias propiedades en el control Button para hacer esto, incluyendo (pero no limitado a) **Foreground** , **Background** , **FontWeight** y así sucesivamente. En otras palabras, es muy fácil cambiar el formato del texto dentro de un control Button:

```
<Button Background="Beige" Foreground="Blue" FontWeight="Bold">Formatted Button</Button>
```

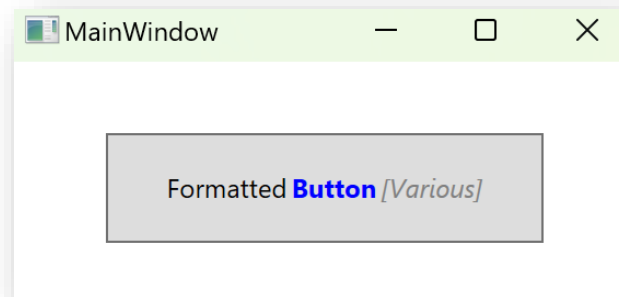


BOTONES CON CONTENIDO AVANZADO

El botón WPF **solo admite un control secundario directo**, pero puede hacer que sea un Panel, que luego albergará tantos controles como sea necesario. Puedes usar esto para crear botones con varios tipos de formato:

```
<Button Height="50" Width="200">
    <StackPanel Orientation="Horizontal">
        <TextBlock>Formatted</TextBlock>
        <TextBlock Foreground="Blue" FontWeight="Bold" Margin="2,0">Button</TextBlock>
        <TextBlock Foreground="Gray" FontStyle="Italic">[Various]</TextBlock>
    </StackPanel>
</Button>
```

Obtendríamos la siguiente salida:



Por supuesto, no estás limitado a usar solo texto - puedes poner lo que quieras dentro de tus botones, incluso imágenes.

BOTONES CON IMÁGENES

En WPF, no hay necesidad de un control separado para incluir una imagen en el botón. Como acabas de ver, podemos colocar varios controles dentro de un botón, de modo que puedes agregarle un control *Image* al botón. Pero antes debemos agregar la imagen al proyecto:

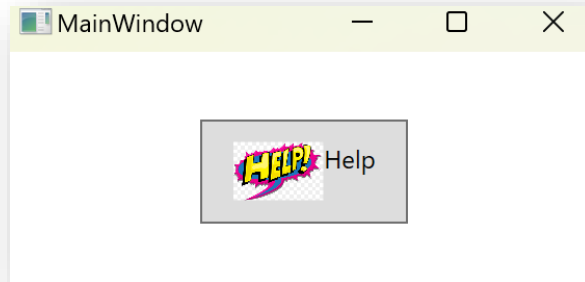
1. Para que no haya problemas con la ubicación, crea un directorio dentro del proyecto que se llame Images por ejemplo, y guarda ahí la imagen que quieres añadir.
2. Nos vamos a nuestro proyecto en Visual Studio, hacemos click con el botón derecho en el proyecto → Agregar → Elemento existente, y seleccionamos nuestra imagen.
3. Ponemos en la característica Source del control Image la ruta donde se aloja la imagen.

NOTA: podemos hacer click con el botón derecho sobre la imagen y nos da la opción de copiar la ruta en la que se ubica.

Una vez hecho esto, escribimos el código de este ejemplo:

```
<Button Padding="5">
  <StackPanel Orientation="Horizontal">
    <Image Source="C:\Users\meli6\source\repos\Wpfccf2\help.png" Height="28" Width="44"/>
    <TextBlock><Run Text="Help"/></TextBlock>
  </StackPanel>
</Button>
```

Y veríamos la siguiente salida al ejecutar:



RELLENO DE UN BOTÓN

- Para separar el contenido del botón de los bordes, existe la propiedad **Padding** que se puede usar como vemos en este ejemplo:

```
<Button Padding="15" Height="50" Width="100">Hello, World!</Button>
```



Establecerá 15 unidades **en todos los lados** del contenido del botón.

-Si solo especificamos dos valores:

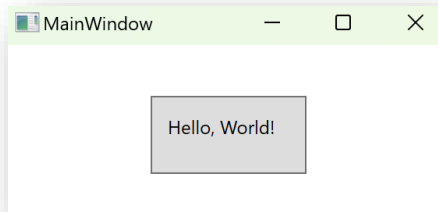
```
<Button Padding="10,5" Height="50" Width="100">Hello, World!</Button>
```



El primer valor se aplica al relleno izquierdo y derecho. El segundo valor se aplica al relleno superior e inferior.

-Si queremos especificar cada lado lo haríamos como en el siguiente ejemplo:

```
<Button Padding="10,5,20,15" Height="50" Width="100">Hello, World!</Button>
```



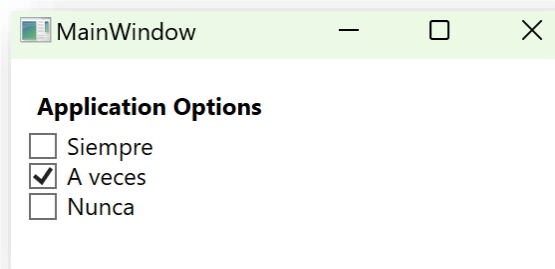
10 unidades para el relleno izquierdo. **5** unidades para el relleno superior. **20** unidades para el relleno derecho. **15** unidades para el relleno inferior.

5.5. El control CHECKBOX

El control CheckBox permite al usuario final alternar una opción entre encendido o apagado. Usualmente refleja un valor Boolean en el code-behind. Vamos a ver el ejemplo:

```
<StackPanel Margin="10">
    <Label FontWeight="Bold">Application Options</Label>
    <CheckBox>Siempre</CheckBox>
    <CheckBox IsChecked="True">A veces</CheckBox>
    <CheckBox>Nunca</CheckBox>
</StackPanel>
```

Obtendríamos:



El CheckBox es muy fácil de usar. En el segundo CheckBox se ha utilizado la propiedad **IsChecked** para **marcarlo por defecto**, pero no son necesarias más propiedades para usarlo. La propiedad **IsChecked** también se puede usar desde el code-behind para determinar si un CheckBox está marcado o no.

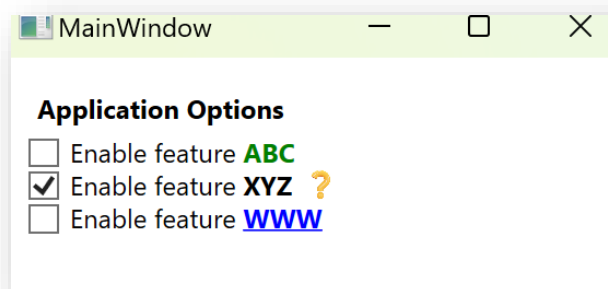
CONTENIDO PERSONALIZADO

En el checkbox se puede utilizar cualquier tipo de control dentro de él, tal y como veremos en el siguiente ejemplo:

```

<StackPanel Margin="10">
  <Label FontWeight="Bold">Application Options</Label>
  <CheckBox>
    <TextBlock> Enable feature <Run Foreground="Green"
                                FontWeight="Bold">ABC</Run></TextBlock>
  </CheckBox>
  <CheckBox IsChecked="True">
    <WrapPanel>
      <TextBlock> Enable feature <Run FontWeight="Bold">XYZ</Run></TextBlock>
      <Image Source="/interrogacion_naranja.jpg" Width="16" Height="16" Margin="5,0" />
    </WrapPanel>
  </CheckBox>
  <CheckBox>
    <TextBlock> Enable feature <Run Foreground="Blue" TextDecorations="Underline"
                                FontWeight="Bold">WWW</Run></TextBlock>
  </CheckBox>
</StackPanel>

```

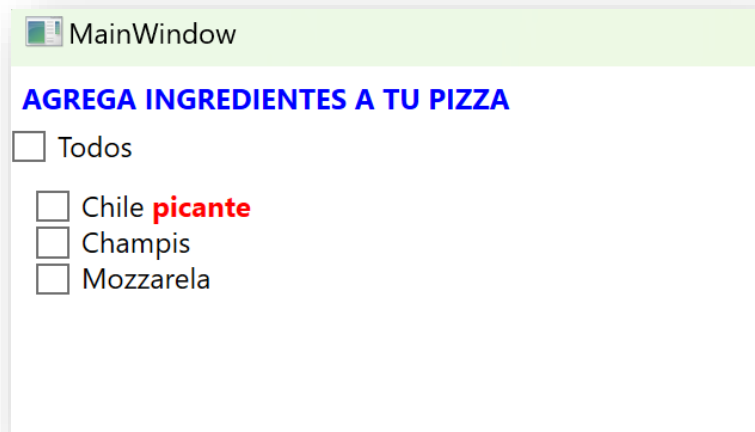


LA PROPIEDAD ISTHREESTATE

Como hemos dicho, el CheckBox normalmente se corresponde con un valor lógico, lo que significa que tiene solo dos estados: true o false (on y off). En cualquier caso, puesto que un tipo de datos boolean podría ser null, permitiendo de hecho una tercera opción (true, false o null) el control CheckBox también permite este escenario. Poniendo la propiedad IsThreeState a true, el CheckBox podrá adquirir el tercer estado llamado "el estado intermedio".

Una utilización habitual para esto es disponer de un CheckBox que "habilite todo", que puede controlar un conjunto de Checkboxes hijos así como mostrar su estado colectivo. Nuestro ejemplo muestra cómo podrías crear una lista de características que pueden cambiarse a encendidas o apagadas, con un CheckBox común que las "habilite todas" encima.

Vemos el siguiente ejemplo, que dará este resultado como aplicación:



1. Ponemos un stackpanel que albergue el primer checkbox y luego otro que tenga los tres checkbox restantes.

```
<StackPanel>
  <Label FontWeight="Bold" Foreground="Blue">AGREGA INGREDIENTES A TU PIZZA</Label>
  <CheckBox Name="cbTodos" IsThreeState="True" Checked="cb_TodosHaCambiado"
    Unchecked="cb_TodosHaCambiado">Todos</CheckBox>
  <StackPanel>
    <CheckBox Name="cbChile" Checked="cb_SimpleHaCambiado"
      Unchecked="cb_SimpleHaCambiado">Chile</CheckBox>
    <CheckBox Name="cbChampis" Checked="cb_SimpleHaCambiado"
      Unchecked="cb_SimpleHaCambiado">Champis</CheckBox>
    <CheckBox Name="cbMozzarella" Checked="cb_SimpleHaCambiado"
      Unchecked="cb_SimpleHaCambiado">Mozzarella</CheckBox>
  </StackPanel>
</StackPanel>
```

Añadimos al checkbox la característica de Name, empezando éste por cb (de checkbox), ponemos la propiedad IsThreeState a true y Checked, que será el evento que se active cuando se marque este check. **El estado marcado y desmarcado llamaran al mismo método para gestionar el evento.**

2. Creamos el método manualmente y le ponemos el nombre indicado arriba.

```
private void cb_TodosHaCambiado(object sender, RoutedEventArgs e)
{
  // ...
}
```

3. Creamos los 3 CheckBox simples con un mismo método para el evento que activa cada uno de ellos. Dentro del cada checkbox ponemos un TextBlock con un <run>para darle formato.

Este sería el primer checkbox:

```
<StackPanel>
  <CheckBox Name="cbChile" Checked="cb_SimpleHaCambiado"
    Unchecked="cb_SimpleHaCambiado">
    <TextBlock>Chile <Run Foreground="Red" FontWeight="Bold">picante</Run></TextBlock>
  </CheckBox>
  <CheckBox Name="cbChampis" Checked="cb_SimpleHaCambiado"
    Unchecked="cb_SimpleHaCambiado">Champis</CheckBox>
  <CheckBox Name="cbMozzarella" Checked="cb_SimpleHaCambiado"
    Unchecked="cb_SimpleHaCambiado">Mozzarella</CheckBox>
</StackPanel>
```

Y su método:

```
private void cb_SimpleHaCambiado(object sender, RoutedEventArgs e)
{
    ...
}
```

Creamos los siguientes checkbox más simples:

```
<StackPanel>
    <Label FontWeight="Bold" Foreground="Blue">AGREGA INGREDIENTES A TU PIZZA</Label>
    <CheckBox Name="cbTodos" IsThreeState="True" Checked="cb_TodosHaCambiado"
        Unchecked="cb_TodosHaCambiado">Todos</CheckBox>
    <StackPanel Margin="10">
        <CheckBox Name="cbChile" Checked="cb_SimpleHaCambiado"
            Unchecked="cb_SimpleHaCambiado">
            <TextBlock>Chile <Run Foreground="Red" FontWeight="Bold">picante</Run></TextBlock>
        </CheckBox>
        <CheckBox Name="cbChampis" Checked="cb_SimpleHaCambiado" Unchecked="cb_SimpleHaCambiado">
            Champis
        </CheckBox>
        <CheckBox Name="cbMozzarella" Checked="cb_SimpleHaCambiado" Unchecked="cb_SimpleHaCambiado">
            Mozzarella
        </CheckBox>
    </StackPanel>
</StackPanel>
```

Hemos agregado una propiedad al StackPanel que contiene los 3 checkbox simples para que se distancien del primer checkbox.

Completamos el código de los dos métodos para que tenga el comportamiento habitual en este tipo de escenarios:

```
private void cb_TodosHaCambiado(object sender, RoutedEventArgs e)
{
    //Valor que tendrá el check cuando se genere el cambio
    bool nuevoValor = (cbTodos.IsChecked == true); //Se cargará con true o false según si está activado
    //Actualizamos los valores de los checkbox según esté el valor de "cbTodos"
    cbChile.IsChecked = nuevoValor;
    cbChampis.IsChecked = nuevoValor;
    cbMozzarella.IsChecked = nuevoValor;
}

0 referencias
private void cb_SimpleHaCambiado(object sender, RoutedEventArgs e)
{
    cbTodos.IsChecked = null; //Lo ponemos en el tercer estado por defecto
    if((cbChile.IsChecked==true)&& (cbChampis.IsChecked == true)&& (cbMozzarella.IsChecked == true))
    {
        cbTodos.IsChecked=true;
    }
    else if ((cbChile.IsChecked == false) && (cbChampis.IsChecked == false)
        && (cbMozzarella.IsChecked == false))
    {
        cbTodos.IsChecked=false;
    }
}
```

5.6. Control RADIOBUTTON

El **RadioButton** se utiliza cuando deseas que el usuario seleccione una única opción dentro de un conjunto de opciones mutuamente exclusivas. En un grupo de **RadioButtons**, solo uno puede estar seleccionado al mismo tiempo.

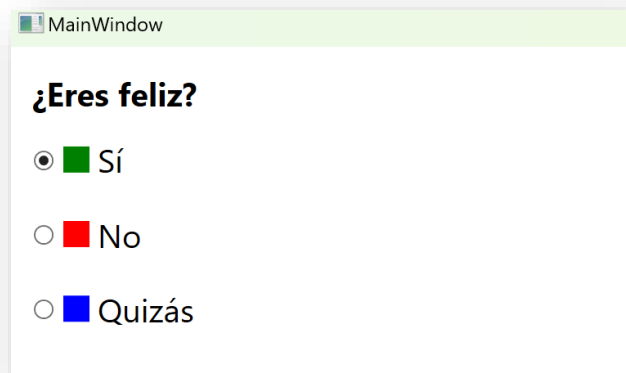
Veamos un ejemplo:

```
<StackPanel Margin="10">
    <Label FontWeight="Bold" FontSize="20">¿Eres feliz?</Label>
    <RadioButton VerticalContentAlignment="Center" Margin="5">
        <WrapPanel>
            <Rectangle Fill="Green" Width="16" Height="16"/>
            <TextBlock Text="Sí" FontSize="20" Margin="5"></TextBlock>
        </WrapPanel>
    </RadioButton>
    <RadioButton VerticalContentAlignment="Center" Margin="5">
        <WrapPanel>
            <Rectangle Fill="Red" Width="16" Height="16"/>
            <TextBlock Text="No" FontSize="20" Margin="5"></TextBlock>
        </WrapPanel>
    </RadioButton>
    <RadioButton VerticalContentAlignment="Center" Margin="5">
        <WrapPanel>
            <Rectangle Fill="Blue" Width="16" Height="16"/>
            <TextBlock Text="Quizás" FontSize="20" Margin="5"></TextBlock>
        </WrapPanel>
    </RadioButton>
</StackPanel>
```

Sobre el código:

- Dentro de RadioButton:
 - **VerticalContentAlignment="Center"**: Alinea verticalmente el contenido del **RadioButton** (en este caso, el **WrapPanel**) en el centro del área disponible del botón de opción. Esto significa que el contenido, como el texto o los controles dentro del **RadioButton**, se colocarán en el centro de su espacio vertical.
 - **Margin="5"**: Establece un espacio de 5 unidades alrededor del **RadioButton**, creando un margen entre este botón y otros elementos adyacentes.
- Dentro del **WrapPanel**, tenemos dos elementos:
 - **Rectangle**→**Fill="Green"**: Establece el color de relleno del **Rectangle** a **verde**. Es el color de la forma del rectángulo.

Al ejecutarlo veríamos lo siguiente:



Solo podremos seleccionar una de las opciones de RadioButton que tengamos en la ventana.

EVENTOS EN RADIOBUTTON

También podemos vincular eventos a la selección. Una forma rápida es haciendo doble click en la ventana en la que estamos haciendo el diseño, y aparecerá tanto en la parte de XAML como en la de C# los elementos necesarios.

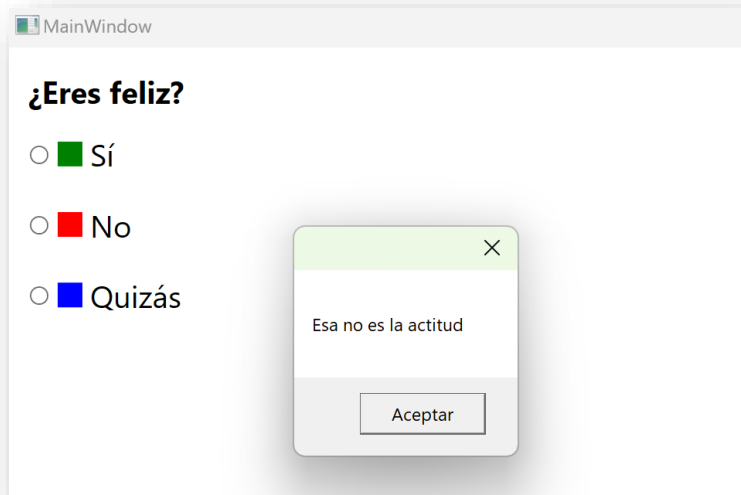
Podemos poner que se maneje el evento cuando seleccionamos la opción “No”. Si cambiamos el nombre del método para que sea más representativo, podría ser algo así:

```
<RadioButton VerticalContentAlignment="Center" Margin="5" Checked="No_Seleccionado">
    <WrapPanel>
        <Rectangle Fill="Red" Width="16" Height="16"/>
        <TextBlock Text="No" FontSize="20" Margin="5"></TextBlock>
    </WrapPanel>
</RadioButton>
```

En la parte de C#:

```
private void No_Seleccionado(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Esa no es la actitud");
}
```

Resultaría lo siguiente al seleccionar el “No”:



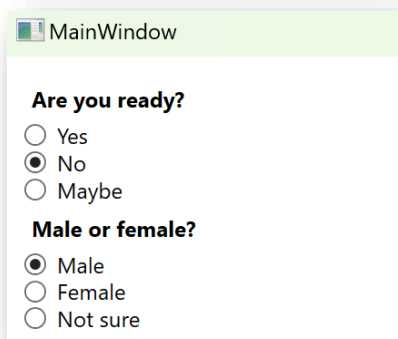
GRUPOS DE RADIOBUTTON

¿Qué pasa si quieres un grupo de radiobuttons y puedan tener cada uno de los grupos una selección individual? Esto es lo que hace la propiedad **GroupName** que te permite especificar que radio button pertenecen a un grupo.

Veamos un ejemplo:

```
<StackPanel Margin="10">
    <Label FontWeight="Bold">Are you ready?</Label>
    <RadioButton GroupName="ready">Yes</RadioButton>
    <RadioButton GroupName="ready">No</RadioButton>
    <RadioButton GroupName="ready" IsChecked="True">Maybe</RadioButton>
    <Label FontWeight="Bold">Male or female?</Label>
    <RadioButton GroupName="sex">Male</RadioButton>
    <RadioButton GroupName="sex">Female</RadioButton>
    <RadioButton GroupName="sex" IsChecked="True">Not sure</RadioButton>
</StackPanel>
```

Darí­a como resultado de su ejecución:



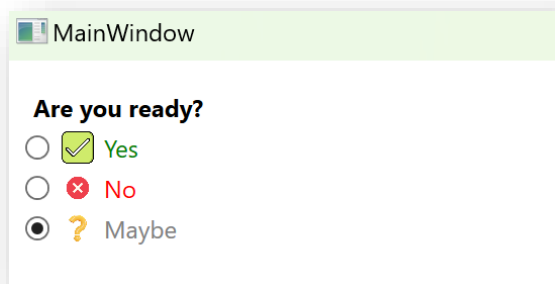
Con la propiedad `GroupName` establecida en cada uno de los radio buttons, ahora se puede hacer una selección para cada uno de los dos grupos. Sin esto, solo sería posible una selección para los seis `RadioButtons`.

CONTENIDO PERSONALIZADO

`RadioButton` hereda de la clase `ContentControl`, lo que significa que puede tomar contenido personalizado y mostrarlo junto a él. Si solo especifica un fragmento de texto, como hicimos en el ejemplo anterior, WPF lo colocará dentro de un control `TextBlock` y lo mostrará, pero esto es solo un atajo para facilitarnos las cosas. Puedes usar cualquier tipo de control dentro de él, como veremos en el siguiente ejemplo:

```
<StackPanel Margin="10">
  <Label FontWeight="Bold">Are you ready?</Label>
  <RadioButton>
    <WrapPanel>
      <Image Source="/aceptar.png" Width="16" Height="16" Margin="0,0,5,0" />
      <TextBlock Text="Yes" Foreground="Green" />
    </WrapPanel>
  </RadioButton>
  <RadioButton Margin="0,5">
    <WrapPanel>
      <Image Source="/rechazar.png" Width="16" Height="16" Margin="0,0,5,0" />
      <TextBlock Text="No" Foreground="Red" />
    </WrapPanel>
  </RadioButton>
  <RadioButton IsChecked="True">
    <WrapPanel>
      <Image Source="/interrogacion_naranja.jpg" Width="16" Height="16" Margin="0,0,5,0" />
      <TextBlock Text="Maybe" Foreground="Gray" />
    </WrapPanel>
  </RadioButton>
</StackPanel>
```

Como resultado tendríamos lo siguiente:



En cuanto a las marcas, este ejemplo se vuelve un poco pesado, pero el concepto es bastante simple. Para cada `RadioButton`, tenemos un `WrapPanel` con una imagen y una pieza de texto dentro de ella. Ya que ahora tomamos el control del texto usando un control `TextBlock`, esto también nos permite formatear el texto de la manera que queramos. Para este ejemplo, se ha cambiado el color del texto para que coincida con la elección. Se utiliza un control de imagen (lee más sobre ellos más adelante) para mostrar una imagen para cada opción.

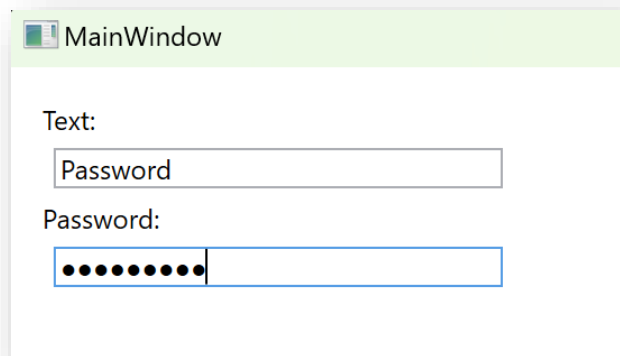
Observa cómo puedes hacer clic en cualquier lugar de `RadioButton`, incluso en la imagen o el texto, para alternar entre ellos, porque lo hemos especificado como contenido de `RadioButton`. Si se hubiera colocado como un panel separado, al lado del `RadioButton`, el usuario tendría que hacer clic directamente en el círculo redondo de `RadioButton` para activarlo, lo cual es menos práctico.

5.7. Control `PASSWORDBOX`

Para editar texto normal, podemos utilizar un `TextBox`. Pero, ¿qué hay de la edición de contraseñas? Buscamos la misma funcionalidad básica, aunque para las contraseñas, deseamos esconder los caracteres ingresados para proteger nuestra contraseña de cualquier metiche que nos esté observando sobre el hombro. Para este propósito, WPF nos provee el control de **`PasswordBox`**, que resulta tan fácil de usar como un `TextBox`. Ilustrémoslo con el siguiente ejemplo:

```
<StackPanel Margin="10">
  <Label>Text:</Label>
  <TextBox Width="200" HorizontalAlignment="Left" Margin="10,0,0,0"/>
  <Label>Password:</Label>
  <PasswordBox Width="200" HorizontalAlignment="Left" Margin="10,0,0,0"/>
</StackPanel>
```

Obtendríamos:



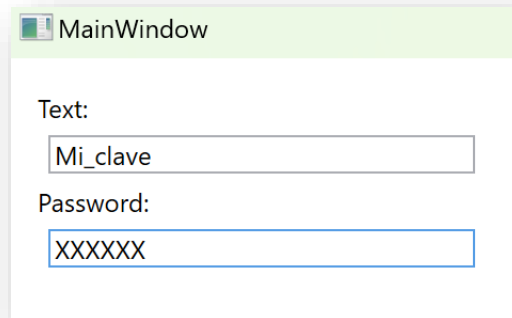
En la imagen, hemos introducido el mismo texto en ambas cajas de texto. Sin embargo, en la caja para la contraseña, los caracteres que hemos ingresado han sido reemplazados con puntos. De hecho, podemos controlar qué carácter aparecerá en lugar de lo que ingresemos, utilizando la propiedad **`PasswordChar`**. Notemos:

```
<PasswordBox PasswordChar="X" />
```

En este caso, una “X” aparecerá en lugar de los puntos vistos anteriormente. Además, si deseamos controlar la longitud de la contraseña, podemos utilizar la propiedad de **`MaxLength`**, como sigue:

`<PasswordBox MaxLength="6" />`

Si añadimos estas dos propiedades al código anterior, quedaría:



6. PROPIEDADES DE DEPENDENCIA

Las **propiedades de dependencia** son un tipo especial de propiedades que ofrecen un conjunto de características avanzadas, como la capacidad de vincularse a otras propiedades, proporcionar valores predeterminados, realizar un seguimiento de cambios automáticamente, y trabajar de manera eficiente con el sistema de diseño de WPF, como la **interfaz de usuario (UI)**.

Veamos un ejemplo a continuación, en el que cuando pasemos por encima sobre un botón, el tamaño de las letras de dicho botón aumenten, y vuelvan a su tamaño original cuando el ratón deje de estar sobre el botón:

```
<Grid>
  <Button Height="100" Width="200" Content="Hola Mundo" Background="PaleVioletRed">
    <Button.Style>
      <Style TargetType="Button">
        <Style.Triggers>
          <Trigger Property="IsMouseOver" Value="True">
            <Setter Property="FontSize" Value="30"/>
          </Trigger>
        </Style.Triggers>
      </Style>
    </Button.Style>
  </Button>
</Grid>
```

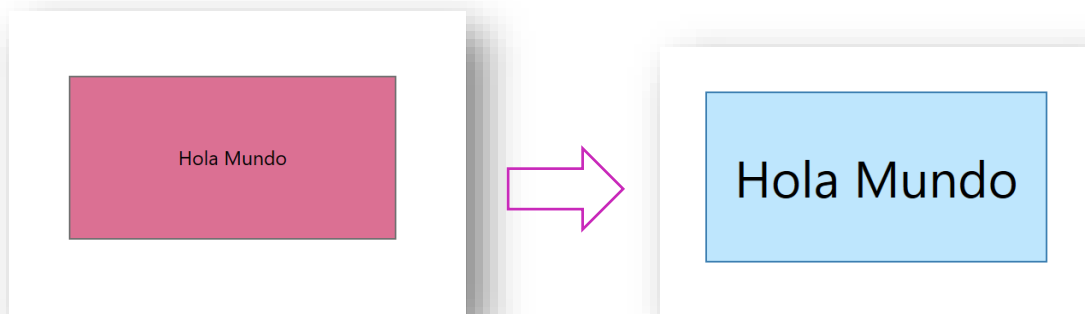
Sobre el código:

- **<Button.Style>**: Define un bloque de estilo para este botón en particular. El estilo es una colección de reglas que permiten personalizar la apariencia y el comportamiento del control.
- **<Style TargetType="Button">**: El estilo está diseñado para aplicarse a controles de tipo Button.

- **<Style.Triggers>**: Los *triggers* (desencadenadores) son condiciones que permiten cambiar dinámicamente propiedades del control cuando se cumple una condición específica. En este caso, los desencadenadores se aplican cuando se detecta un cambio en la propiedad del botón.
- **<Trigger Property="IsMouseOver" Value="True">**: Este es el desencadenador en sí. Se activa cuando la propiedad IsMouseOver del botón es True, es decir, cuando el ratón está sobre el botón.
 - **Property="IsMouseOver"**: Especifica la propiedad del control que se está monitorizando. En este caso, IsMouseOver es una propiedad booleana que indica si el ratón está encima del botón.
 - **Value="True"**: La condición es que IsMouseOver debe ser verdadero (el ratón está sobre el botón).
- **<Setter Property="FontSize" Value="30"/>**: Este bloque de código define lo que sucede cuando el desencadenador se activa. En este caso, cuando el ratón está sobre el botón (IsMouseOver = True), la propiedad FontSize del botón se establece a 30.

En este ejemplo se utilizan varias propiedades de dependencia, como Height, Width, Content, Background, FontSize, IsMouseOver, y Style.

Daríá como resultado de ejecución lo siguiente:



Si queremos ver cómo es la estructura de clases con la que estamos trabajando, vamos al archivo de C#, simplemente escribimos la palabra Button y dando al botón derecho, pulsamos en “Ir a definición”:

```
public partial class MainWindow : Window
{
    0 referencias
    public MainWindow()
    {
        InitializeComponent();
        Button
    }
}
```

Si hacemos lo mismo en **ButtonBase** (la clase de la que hereda), posteriormente repetimos con **ContentControl** y finalmente con **Control**, vemos como entre los componentes de la clase está la propiedad de dependencia **FontSizeProperty** con la que hemos trabajado:

```
...public static readonly DependencyProperty FontSizeProperty;
```

Y más abajo la propiedad **FontSize**:

```
public double FontSize
{
    get { ... }
    set { ... }
}
```

FontSize es la propiedad pública que se expone para que los desarrolladores puedan leer o escribir el tamaño de la fuente en el control.

FontSizeProperty es la propiedad de dependencia que realmente maneja y almacena el valor de **FontSize**. Es una propiedad interna que permite a WPF aprovechar el sistema de propiedades de dependencia, proporcionando características avanzadas como la vinculación de datos, la herencia de valores y la optimización del rendimiento.

6.1. DependencyProperty personalizadas

Lo haremos de la siguiente manera:

1. Creamos la propiedad pública:

```
0 referencias
public int ColorDeOjos { get; set; }
0 referencias
public MainWindow()
{
    InitializeComponent();
}
```

2. Creamos la propiedad estática, la que va a almacenar a nivel de sistema y modificarse cuando cambie la anterior:

```
//Propiedad pública
0 referencias
public int ColorDeOjos { get; set; }
//Propiedad estática DependencyProperty
public static readonly DependencyProperty ColorDeOjosProperty=
    DependencyProperty.Register("ColorDeOjos",typeof(int),typeof(MainWindow),
        new PropertyMetadata(0));
```

La registramos en el constructor con 4 parámetros:

- Nombre: "ColorDeOjos"
- Tipo: int
- Tipo de propietario: MainWindow
- Tipo de metadatos (que va a tener por detrás)

Para relacionar ambas propiedades completamos los métodos get y set:

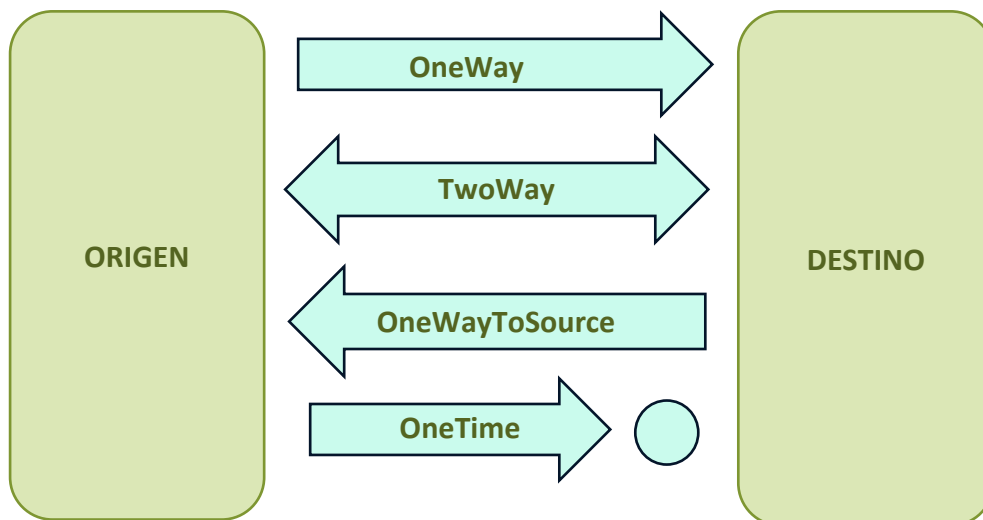
```
public int ColorDeOjos {
    get { return (int)GetValue(ColorDeOjosProperty); }
    set { SetValue(ColorDeOjosProperty, value); }
}
```

7. ENLACE DE DATOS: DATABINDING

En WPF, **DataBinding** se basa en la propiedad de dependencia, lo que significa que los valores de la UI pueden actualizarse dinámicamente cuando los valores de las propiedades subyacentes cambian.

Cuando se establece un binding, WPF crea una conexión entre una propiedad en la interfaz de usuario y una propiedad en el modelo de datos.

Hay tres formas, según la direccionalidad del enlace:



- OneWay: el enlace de datos va desde el origen al destino, terminando en este momento.
- TwoWay: puede ir del origen al destino y viceversa.
- OneWayToSource: el enlace va desde el destino al origen.
- OneTime: se desdende el origen al destino pero solo sucede una vez y deja de establecerse la conexión si se modifican los datos.

Veamos un ejemplo para ver su uso:

1. Creamos un botón y un <Slider> dentro de un StackPanel:

```
<StackPanel>
    <TextBox Width="100" Margin="50" Text="15"/>
    <Slider Name="MiSlider" Minimum="0" Maximum="100"/>
</StackPanel>
```

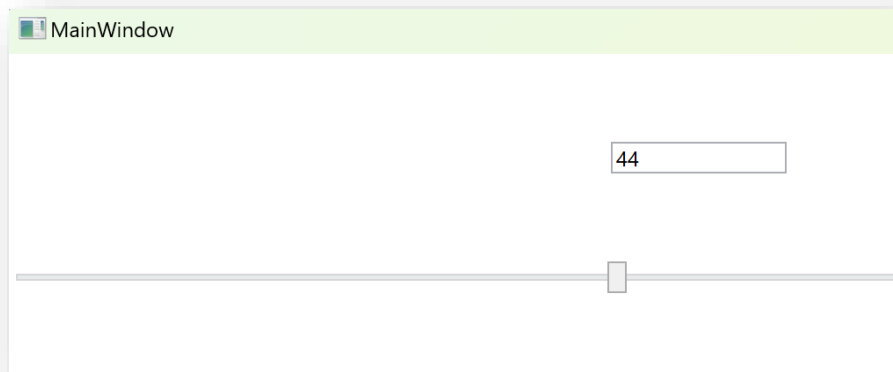
2. Ahora los enlazamos con "Binding" y especificamos qué van a compartir en "Path" y de qué manera con "Mode":

```
<StackPanel>
    <TextBox Width="100" Margin="50" Text="{Binding ElementName=MiSlider,Path=Value,Mode=OneWay}"/>
    <Slider IsSnapToTickEnabled="True" Name="MiSlider" Minimum="0" Maximum="100"/>
</StackPanel>
```

En este caso, cuando ponemos el Binding, el elemento que lo contiene es el destino, con lo cual, el origen sería el Slider y el destino el TextBox.

La propiedad IsSnapToTickEnabled sirve para que redondee a la unidad más alta que tenga cerca. La añadimos a true para que se vea mejor el comportamiento.

Ejecutamos:



- Si movemos la barra: origen, se cambia el valor del textbox, pero no viceversa.
- Si cambiamos el modo a **ToWay** y repetimos operación, tras cambiar el valor del TextBox, y pulsamos Tab, se cambiará el desplazamiento de la barra.
- Si cambiamos a **OneWayToSource**, vemos que el comportamiento es contrario que el OneWay.
- Ponemos finalmente la opción **OneTime**. Esta propiedad hay que establecerla en C# con lo que agregamos lo siguiente:
 - o Nombre al TextBox → Name="MiTextBox"
 - o El siguiente código en C#:

```

0 referencias
public MainWindow()
{
    InitializeComponent();
    MiSlider.Value = 30;
    MiTextBox.Text=MiSlider.Value.ToString();
}

```

Si lo ejecutamos, se establece el mismo valor de 30 para el TextBox que para la barra, pero al modificarlos, ninguno de ellos reacciona al cambio del otro.

- Finalmente vamos a realizar una variación en el modo TwoWay, añadiendo la propiedad UpdateSourceTrigger para que cuando se produzca un cambio en el TextBox, se visualice esta modificación en el otro elemento sin tener que pulsar tab:

```

<StackPanel>
    <TextBox Name="MiTextBox" Width="100" Margin="50" Text="{Binding ElementName=MiSlider,
        Path=Value,Mode=TwoWay,UpdateSourceTrigger=PropertyChanged}"/>
    <Slider IsSnapToTickEnabled="True" Name="MiSlider" Minimum="0" Maximum="100"/>
</StackPanel>

```

Si cambiamos el parámetro de **Path** podremos compartir otros cambios, como por ejemplo, poner Path=BackGround y añadir la propiedad BackGround al Slider, poniéndole por ejemplo Aqua.

En este caso, se escribiría en el TextBox el valor en hexadecimal del color de la barra.



7.1. Binding con propiedades de dependencia personalizada

En el ejemplo anterior para vincular el TextBox con el Slider hemos utilizado la propiedad ElementName, pero no será la opción adecuada si estamos haciendo binding con propiedades creadas por nosotros.

¿ELEMENTNAME Y DATACONTEXT?

1. ElementName:

- Permite enlazar propiedades a **otro elemento visual** (por ejemplo, un control como Button, TextBox, etc.) dentro del mismo árbol visual.
- WPF busca en el árbol lógico del XAML un elemento con el nombre proporcionado en ElementName.

2. DataContext:

- Es el contexto de datos que proporciona los valores a las propiedades enlazadas mediante Binding.
- En este caso, DataContext es el Window y contiene la propiedad TamañoTexto.

En resumen:

- **ElementName** → conecta controles entre sí.
- **DataContext** → conecta controles con tus datos (incluye tus Dependency Properties).

Cuando tú creas propiedades (DP), esas propiedades son **datos**, por tanto usan **DataContext**, no *ElementName*.

Ejemplo: usando **DataContext** para enlazar una propiedad personalizada (*Dependency Property*) llamada TamañoTexto a un botón cuyo FontSize depende de esa propiedad.

XAML:

```
<Window x:Class="WpfApp1.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Ejemplo DP + DataContext" Height="200" Width="300">

    <Grid>
        <!-- El botón usa la propiedad de dependencia TamañoTexto
             que está en el DataContext (la propia ventana) -->
        <Button Content="¡Haz clic aquí!"
                FontSize="{Binding TamañoTexto}"
                Width="200" Height="100"
                HorizontalAlignment="Center" VerticalAlignment="Center"
                Click="Button_Click"/>
    </Grid>
</Window>
```

C#:

```
using System.Windows;

namespace WpfApp1
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
            this.DataContext = this;
        }

        public int TamañoTexto
        {
            get { return (int)GetValue(TamañoTextoProperty); }
            set { SetValue(TamañoTextoProperty, value); }
        }

        public static readonly DependencyProperty TamañoTextoProperty =
            DependencyProperty.Register(
                "TamañoTexto",
                typeof(int),
                typeof(MainWindow),
                new PropertyMetadata(20));

        private void Button_Click(object sender, RoutedEventArgs e)
        {
            TamañoTexto += 2;
        }
    }
}
```

¿Qué está pasando aquí?

`this.DataContext = this;`

Le dice a WPF:

“Mis propiedades personalizadas están en la propia ventana”.

En XAML, el botón hace:

`FontSize="{Binding TamañoTexto}"`

Como el DataContext es la ventana, WPF busca esta propiedad allí.

TamañoTexto es una Dependency Property, así que WPF la puede observar y actualizar automáticamente.

Cada vez que haces clic, el botón aumenta su tamaño.

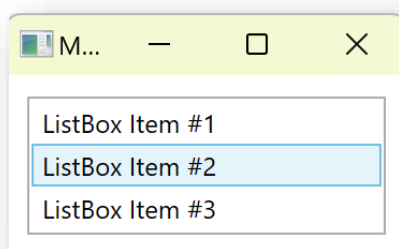
“Ponemos DataContext = this para que la UI pueda acceder a nuestras propiedades personalizadas. Así el enlace FontSize="{Binding TamañoTexto}" funciona sin necesidad de ElementName, porque los datos vienen del DataContext.”

7.2. El control LISTBOX

Este control, que ya hemos comentado anteriormente, trabaja con selecciones, permitiendo al usuario final seleccionar uno o varios elementos de una lista y automáticamente da retroalimentación visual.

A continuación vemos un ejemplo sencillo:

```
<Window x:Class="Wpfccf2.MainWindow"
...
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
xmlns:local="clr-namespace:Wpfccf2"
mc:Ignorable="d"
Title="MainWindow" Height="120" Width="200">
  <Grid Margin="10">
    <ListBox>
      <ListBoxItem>ListBox Item #1</ListBoxItem>
      <ListBoxItem>ListBox Item #2</ListBoxItem>
      <ListBoxItem>ListBox Item #3</ListBoxItem>
    </ListBox>
  </Grid>
</Window>
```



Ahora llevaremos a cabo un ejemplo más complejo, de una aplicación con los resultados y progreso de varios partidos de fútbol:

1. Definimos la estructura:

```
<Grid>
  <!--Definimos las 2 columnas dentro del grid-->
  <Grid.ColumnDefinitions>
```

```

        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="100" />
    </Grid.ColumnDefinitions>
    <!--Creamos el ListBox para agrupar los encuentros-->
    <ListBox Name="lbJuego" Grid.Column="0"
HorizontalContentAlignment="Stretch">
        <ListBox.ItemTemplate>
            <DataTemplate>
                <Grid Margin="2"><!--Creamos las 5 columnas dentro de
este grid-->
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*" />
                        <ColumnDefinition Width="*" />
                        <ColumnDefinition Width="*" />
                        <ColumnDefinition Width="*" />
                        <ColumnDefinition Width="*" />
                    </Grid.ColumnDefinitions>
                </Grid>
            </DataTemplate>
        </ListBox.ItemTemplate>
    </ListBox>
</Grid>

```

Sobre el código:

- **HorizontalContentAlignment="Stretch"** asegura que los elementos dentro del ListBox se estiren para llenar el ancho disponible de la columna.
- El **ItemTemplate** es una propiedad que le permite definir cómo se verá cada elemento dentro del ListBox.
Si no especificamos un ItemTemplate, WPF usará un formato predeterminado para mostrar cada elemento (por ejemplo, mostrando solo el texto si el elemento es una cadena de texto). Pero si deseas un formato más complejo (como mostrar varios controles dentro de cada elemento, o un diseño específico), necesitas definir cómo se verá ese elemento con un ItemTemplate.
- El **ItemTemplate** usa un DataTemplate para describir cómo se debe presentar visualmente cada elemento. **El DataTemplate actúa como una plantilla visual que se aplica a cada ítem** del ListBox.
- Dentro del DataTemplate, se está creando un **Grid con 5 columnas** (definidas por las líneas de código de la columna), lo cual permite tener un diseño estructurado para cada elemento de la lista.
Es adecuado para layouts complejos donde los controles necesitan estar alineados o distribuidos de manera específica.

2. A continuación vamos a poner los TextBlock que aparecerán en cada espacio, al mismo nivel que `</Grid.ColumnDefinitions>`:

```

</Grid.ColumnDefinitions>
<TextBlock Grid.Column="0" Text="{Binding Eq1}"></TextBlock>

```

3.Creamos la clase Eq1 (equipo 1) en C# con la que vamos a vincular este bloque de texto, que se ocupará de los juegos, equipos, puntuaciones y duración. También la lista que va a contener cada Juego:

```
public partial class MainWindow : Window
{
    public MainWindow()
    {
        InitializeComponent();
        List<Juego> juegos = new List<Juego>();
        juegos.Add(new Juego() {Eq1="Barcelona",Eq2="Real Madrid",
                                Puntaje1=3, Puntaje2=2, Progreso=85 });
    }
}
public class Juego
{
    public int Puntaje1 { get; set; }
    public int Puntaje2 { get; set; }
    public string Eq1 { get; set; }
    public string Eq2 { get; set; }
    public int Progreso { get; set; }
}
```

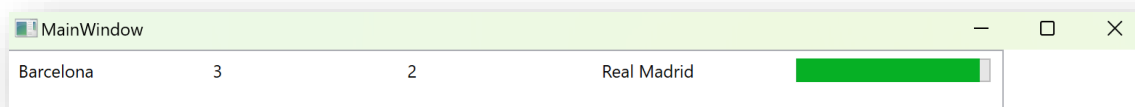
4.Añadimos el resto de TextBox y también la barra de progreso con el elemento <ProgressBar>.

```
<TextBlock Grid.Column="0" Text="{Binding Eq1}"></TextBlock>
<TextBlock Grid.Column="1" Text="{Binding Puntaje1}"></TextBlock>
<TextBlock Grid.Column="2" Text="{Binding Puntaje2}"></TextBlock>
<TextBlock Grid.Column="3" Text="{Binding Eq2}"></TextBlock>
<ProgressBar Grid.Column="4" Minimum="0" Maximum="90" Value="{Binding
Progreso}"/>
```

5.Vinculamos la lista creada en XAML con su fuente de datos, en el fichero .cs:

```
public MainWindow()
{
    InitializeComponent();
    List<Juego> juegos = new List<Juego>();
    juegos.Add(new Juego() {Eq1="Barcelona",Eq2="Real Madrid",
                            Puntaje1=3, Puntaje2=2, Progreso=85 });
    //Vinculamos la lista creada en XAML con su fuente de datos
    lbJuego.ItemsSource = juegos;
}
```

Si lo ejecutamos, nos va quedando así:



Añadimos esta propiedad a los puntajes para que se vea mejor: `HorizontalAlignment="Center"`, si modificamos el tamaño de la ventana.

6. Agregamos 3 juegos más:

```
juegos.Add(new Juego() {Eq1="Barcelona",Eq2="Real Madrid", Puntaje1=2,
Puntaje2=3, Progreso=85 });
juegos.Add(new Juego() { Eq1 = "PSG", Eq2 = "Bayern Munich", Puntaje1 = 2,
Puntaje2 = 3, Progreso = 55 });
juegos.Add(new Juego() { Eq1 = "Málaga", Eq2 = "Sevilla", Puntaje1 = 5,
Puntaje2 = 1, Progreso = 25 });
juegos.Add(new Juego() { Eq1 = "Manchester", Eq2 = "Paris Sant Germain ",
Puntaje1 = 3, Puntaje2 = 1, Progreso = 15 });
```

7. Añadimos el botón de la columna de la derecha, que nos daba más información sobre el partido seleccionado, en XAML, debajo de la etiqueta <ListBox>:

```
<Button Grid.Column="1" Click="Button_Click">
    MÁS INFORMACIÓN
</Button>
```

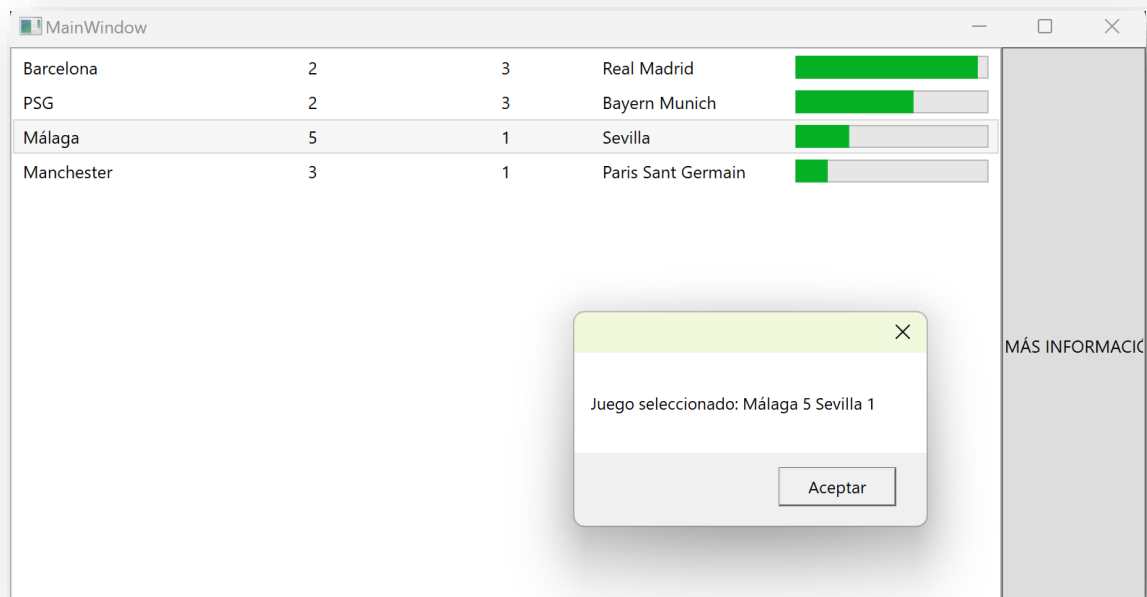
Ahora en C# configuramos el método que gestiona el evento click:

```
private void Button_Click(object sender, RoutedEventArgs e)
{
    if (lbJuego.SelectedItem != null)
    {
        MessageBox.Show("Juego seleccionado: "+(lbJuego.SelectedItem as
Juego).Eq1+
        (lbJuego.SelectedItem as Juego).Puntaje1+ (lbJuego.SelectedItem
as Juego).Eq2
        + (lbJuego.SelectedItem as Juego).Puntaje2);
    }
}
```

SelectedItem no es una clase, es un objeto, pero en este caso lo usamos como una instancia de la clase juego, de la que elegimos la propiedad que queremos ver:

NOTA: si queremos que el tamaño del texto se adapte al espacio disponible podemos poner esto:

```
<TextBlock Text="MÁS INFORMACIÓN"
    TextWrapping="Wrap"
    TextAlignment="Center"/>
```

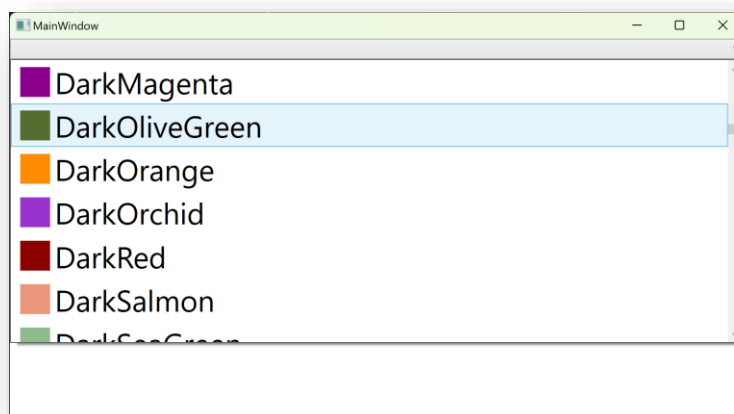
Resumen:

- Hemos creado una clase, Juego, que solo contiene propiedades, lo que es muy “limpio” y práctico en cuanto a código, a la hora de extenderlo.
- Hemos construido la estructura de la aplicación en XAML, y relacionado a través del Binding nuestros elementos con una propiedad de la clase, sin usar la propiedad Path.

7.3. El control COMBOBOX

El control **ComboBox** en WPF es un elemento de interfaz de usuario que permite al usuario seleccionar un elemento de una lista desplegable. Es útil cuando se quiere ofrecer múltiples opciones, pero sin ocupar demasiado espacio en la ventana, ya que la lista completa solo aparece cuando el usuario hace clic en el control.

Ejemplo: ventana en la que se pueda acceder a todos los colores del sistema



1. Generamos el código de la estructura en XAML:

```
<StackPanel>
    <ComboBox Name="ComboBoxColores">
        <ComboBox.ItemTemplate>
            <DataTemplate>
                <StackPanel Orientation="Horizontal">
                    <Rectangle Fill="{Binding Name}" Width="32"
Height="32" Margin="5"/>
                    <TextBlock Text="{Binding Name}" FontSize="32"/>
                </StackPanel>
            </DataTemplate>
        </ComboBox.ItemTemplate>
    </ComboBox>
</StackPanel>
```

2. Configuramos la propiedad Name, con la que se vincularán los dos elementos que hemos definido en el UI, <Rectangle> y <TextBlock>.

```
public MainWindow()
{
    InitializeComponent();
    //Asociamos la fuente de información que se va a cargar en
nuestro ComboBoxColores
    ComboBoxColores.ItemsSource = typeof(Colors).GetProperties();
}
```

Sobre el código:

- **typeof(Colors)**
 - El tipo Colors pertenece al espacio de nombres System.Windows.Media y contiene propiedades estáticas que representan colores predefinidos (por ejemplo: Red, Blue, Green, etc.).
 - Aquí se obtiene el tipo Colors como un objeto Type que permite realizar reflexiones (inspección de metadatos).
- **GetProperties()**
 - Este método, perteneciente a la clase Type, devuelve una lista de todas las propiedades públicas de la clase Colors.
 - En este caso, devolverá un arreglo de objetos PropertyInfo, donde cada uno representa una propiedad estática de color, como Red, Blue, Green, etc.

En el ejemplo anterior del ListBox, la fuente de información del elemento era una lista de objetos de la clase Juegos, que habíamos creado. En este caso, será información que ya tiene el sistema.

8. TOOLTIP: INFORMACIÓN SOBRE HERRAMIENTAS

En **WPF**, un **Tooltip** es un cuadro emergente que aparece cuando el usuario coloca el puntero del ratón sobre un elemento de la interfaz de usuario (como un botón, una etiqueta o cualquier control). Se utiliza para proporcionar información adicional o contextualmente relevante sobre ese elemento.

Veamos un ejemplo: creamos un botón al que le ponemos información adicional.

```
<Grid>
  <Button Width="150" Height="100" Content="Pulsa aquí" ToolTip="Estás
sobre el botón"/>

</Grid>
```

Ahora veremos como se haría con un TextBox:

```
<Grid>
  <TextBox Width="100" ToolTip="Introduzca su edad"
HorizontalAlignment="Center"
  VerticalAlignment="Center"
TextAlignment="Center">Edad</TextBox>
</Grid>
```

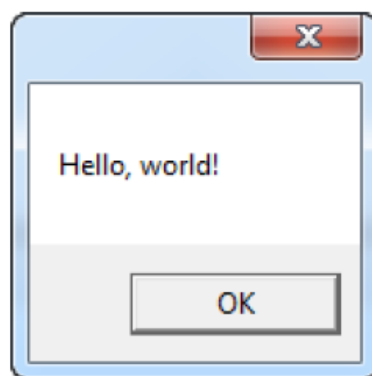
9. MESSAGEBOX

MessageBox se utiliza llamando al método estático Show(), que puede recibir una gama de diferentes parámetros para ser capaz de verse y comportarse de la forma que desees. Veremos todas las diversas formas en este artículo, con cada variación representada por la línea MessageBox.Show() y una captura de pantalla del resultado.

Creemos un botón sencillo cuyo evento Click se maneje mostrando un mensaje por pantalla a través de MessageBox.

En su forma más sencilla MessageBox solamente recibe un parámetro, que es el mensaje que se mostrará:

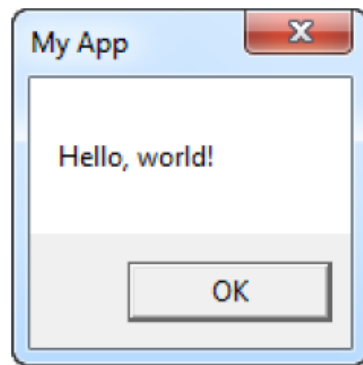
```
MessageBox.Show("Hello, world!");
```



9.1. MESSAGEBOX con título

El ejemplo anterior podría ser un poco básico; un título en la ventana que está mostrando el mensaje probablemente sea útil. Afortunadamente, el segundo parámetro opcional nos permite especificar el título:

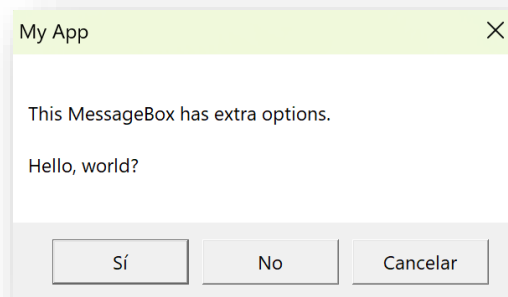
```
MessageBox.Show("Hello, world!", "My App");
```



9.2. MESSAGEBOX CON BOTONES EXTRA

Por defecto MessageBox solo tiene el botón de Ok, pero esto puede ser modificado, en caso de que quieras realizar una pregunta a tu usuario y no solo mostrarle algo de información. También observa cómo se utilizan varias líneas en este mensaje utilizando el carácter de salto de línea (\n):

```
MessageBox.Show("This MessageBox has extra options.\n\nHello, world?", "My App",  
MessageBoxButton.YesNoCancel);
```



Se pueden usar los siguientes valores:

- OK
- OKCancel
- YesNoCancel
- YesNo

Ahora, con múltiples opciones, necesitas una forma de poder ver lo que el usuario eligió y afortunadamente, el método `MessageBox.Show()` siempre devuelve un valor de el enumerador **MessageBoxResult** que puedes usar. Aquí hay un ejemplo en el que ponemos el siguiente código en el método que maneja el evento Click:

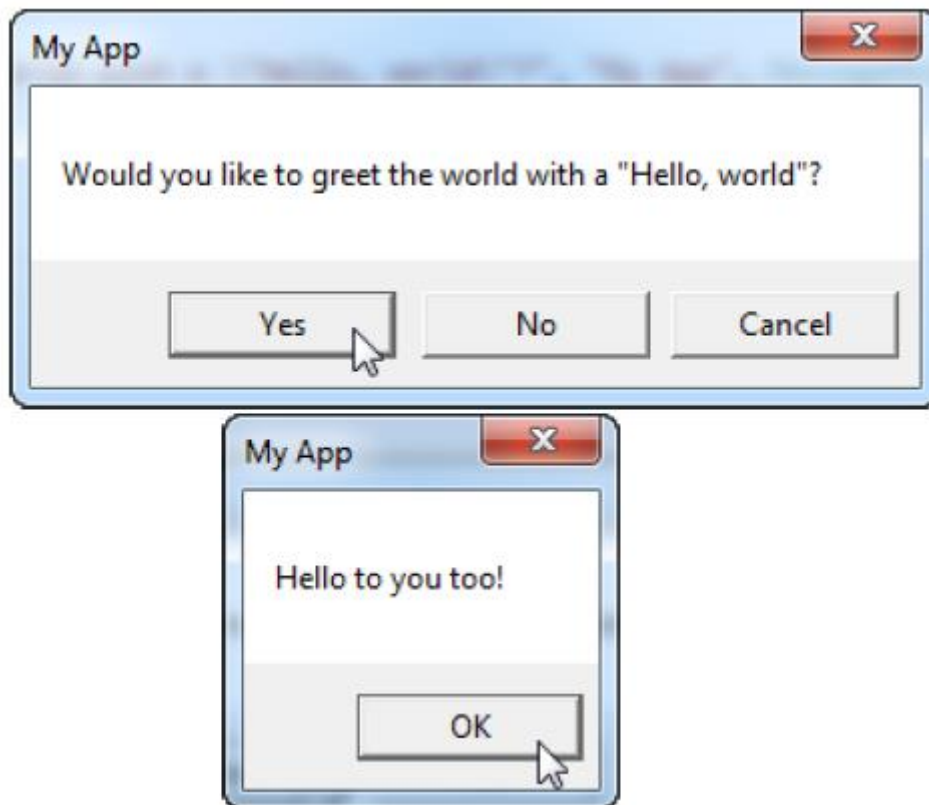
```
private void Button_Click(object sender, RoutedEventArgs e)  
{
```

```

    MessageBoxResult result = MessageBox.Show("Would you like to greet the
world with " +
    "a \"Hello, world\"?", "My App", MessageBoxButton.YesNoCancel);
    switch (result)
    {
        case MessageBoxResult.Yes: MessageBox.Show("Hello to you too!", "My
App");
            break;
        case MessageBoxResult.No: MessageBox.Show("Oh well, too bad!", "My
App");
            break;
        case MessageBoxResult.Cancel: MessageBox.Show("Nevermind then...",
"My App");
            break;
    }
}

```

Resultará lo siguiente:

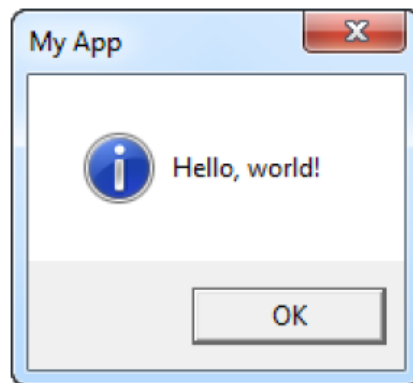


Al verificar el valor del resultado del método `MessageBox.Show ()`, ahora puedes reaccionar ante la elección del usuario, como se ve en el ejemplo del código y en las capturas de pantalla.

9.3. MESSAGEBOX con un icono

`MessageBox` tiene la capacidad de mostrar un icono predefinido a la izquierda del mensaje de texto, utilizando un cuarto parámetro:

```
MessageBox.Show("Hello, world!", "My App", MessageBoxButton.OK,  
MessageBoxImage.Information);
```



Usando el enumerador **MessageBoxImage** , puedes elegir entre un rango de iconos para diferentes situaciones. Esta es la lista completa:

Asterisk (Asterisco)

Error

Exclamation (Exclamación)

Hand (Mano)

Information (información)

None (Ninguno)

Question (Pregunta)

Stop (Alto)

Warning (Advertencia)

9.4. MESSAGEBOX con una opción por defecto

MessageBox seleccionará un botón como la elección predeterminada, que luego será el botón invocado en caso de que el usuario simplemente presione Enter una vez que se muestra el cuadro de diálogo. Por ejemplo, si muestras un MessageBox con botón "Yes" y "No", "Yes" será la respuesta predeterminada. Sin embargo, puedes modificar este comportamiento utilizando un quinto parámetro en el método `MessageBox.Show()`:

```
MessageBox.Show("Hello, world?", "My App", MessageBoxButton.YesNo,  
MessageBoxImage.Question, MessageBoxResult.No);
```

Observa en la captura de pantalla como el botón "No" está ligeramente resaltado, para indicar visualmente que está seleccionado y será invocado si las teclas **Enter** o **Espacio** son presionadas.

10. ANEXO I: EJEMPLO COMPLETO DE EVENTO EN C#

Clase Alarma con Evento AlarmaActivada

Imagina que tenemos una alarma que se activa manualmente mediante un método. Cada vez que se activa, notifica a sus suscriptores con el mensaje "¡La alarma ha sido activada!".

Paso 1: Definir el Delegado

Un **delegado** es como una plantilla de método, que especifica qué tipo de métodos pueden manejar el evento. Para este ejemplo, definiremos un delegado llamado [AlarmaActivadaHandler](#) que aceptará dos parámetros: el objeto que dispara el evento y un conjunto de argumentos ([EventArgs](#)), que es la convención para los eventos.

```
public delegate void AlarmaActivadaHandler(object sender, EventArgs e);
```

Este delegado indica que cualquier método que maneje el evento [AlarmaActivada](#) debe aceptar dos parámetros: un [object](#) (el emisor del evento) y [EventArgs](#).

Paso 2: Crear la Clase Alarma y Definir el Evento

La clase Alarma tendrá un evento llamado [AlarmaActivada](#), basado en el delegado [AlarmaActivadaHandler](#). Cualquier objeto que quiera saber cuándo se activa la alarma puede suscribirse a este evento.


```

using System;

public class Alarma
{
    public delegate void AlarmaActivadaHandler(object sender, EventArgs e);
    // Declaramos el evento, basado en el delegado AlarmaActivadaHandler
    public event AlarmaActivadaHandler AlarmaActivada;

    // Método para activar la alarma
    public void Activar()
    {
        Console.WriteLine("Activando alarma...");
        // Llamamos al método OnAlarmaActivada para disparar el evento
        OnAlarmaActivada();
    }

    // Método protegido para disparar el evento
    protected virtual void OnAlarmaActivada()
    {
        // Verificamos si hay suscriptores al evento antes de dispararlo
        if (AlarmaActivada != null)
        {
            AlarmaActivada(this, EventArgs.Empty); // Disparar el evento
        }
    }
}

```

Explicación del Código:

- **Declaración del Evento:** `public event AlarmaActivadaHandler AlarmaActivada;` es el evento que se activará cuando la alarma se dispare. Otros objetos pueden suscribirse a este evento para que sus métodos se ejecuten cada vez que el evento ocurra.
- **Método Activar:** Este método activa la alarma y llama al método `OnAlarmaActivada`, que dispara el evento.
- **Método OnAlarmaActivada:** Es el método que realmente dispara el evento. Primero, verifica si hay suscriptores (`if (AlarmaActivada != null)`). Si los hay, dispara el evento usando `AlarmaActivada(this, EventArgs.Empty);`, enviando como argumentos:
 - `this`: Se refiere al objeto Alarma que está disparando el evento.

- [EventArgs.Empty](#): No enviamos datos adicionales, así que usamos EventArgs.Empty como un conjunto de argumentos vacío.

Paso 3: Crear una Clase Suscriptor para Escuchar el Evento

Ahora creamos una clase Suscriptor que responderá al evento AlarmaActivada de la alarma.

```
public class Suscriptor
{
    // Método que maneja el evento AlarmaActivada
    public void ManejarAlarmaActivada(object sender, EventArgs e)
    {
        Console.WriteLine("¡La alarma ha sido activada! El suscriptor ha recibido la notificación");
    }
}
```

Este método [ManejarAlarmaActivada](#) cumple con la firma del delegado [AlarmaActivadaHandler](#) (recibe un object y EventArgs) y simplemente muestra un mensaje en la consola.

Paso 4: Configurar el Evento en Main

En el método Main, creamos una instancia de Alarma y una instancia de Suscriptor. Luego, suscribimos el método [ManejarAlarmaActivada](#) de Suscriptor al evento [AlarmaActivada](#) de Alarma.

```

using System;

class Program
{
    static void Main()
    {
        // Crear una instancia de Alarma
        Alarma alarma = new Alarma();

        // Crear una instancia de Suscriptor
        Suscriptor suscriptor = new Suscriptor();

        // Suscribir el método ManejarAlarmaActivada al evento AlarmaActivada
        alarma.AlarmaActivada += suscriptor.ManejarAlarmaActivada;

        // Activar la alarma
        alarma.Activar();
    }
}

```

Explicación del Código:

- `alarma.AlarmaActivada += suscriptor.ManejarAlarmaActivada;`: Esta línea suscribe el método `ManejarAlarmaActivada` de `suscriptor` al evento `AlarmaActivada` de `alarma`.
- `alarma.Activar();`: Llama al método `Activar` en `alarma`, lo cual dispara el evento `AlarmaActivada` y ejecuta todos los métodos suscritos al evento, en este caso, `ManejarAlarmaActivada`.

Ejecución completa del Programa

Cuando el programa se ejecuta, el flujo es el siguiente:

1. D.
2. Suscriptor se suscribe al evento `AlarmaActivada` de `Alarma`.
3. Se llama al método `Activar()` en `Alarma`.
 - Este método imprime "Activando alarma...".
 - Llama al método `OnAlarmaActivada`, que dispara el evento `AlarmaActivada`.

4. Como Suscriptor está suscrito al evento **AlarmaActivada**, el método **ManejarAlarmaActivada** de Suscriptor se ejecuta, mostrando el mensaje: "¡La alarma ha sido activada! El suscriptor ha recibido la notificación."

Resumen:

Flujo de un evento en C#:

1. **Delegado:** Define la firma de los métodos que pueden suscribirse al evento.
2. **Evento:** Se declara en una clase para que otros objetos se suscriban y respondan cuando se dispare.
3. **Método On<Evento>:** Este método verifica si hay suscriptores y dispara el evento si los hay.
4. **Suscriptor:** Es una clase que contiene un método para manejar el evento.
5. **Suscripción:** La clase Suscriptor se suscribe al evento, y cuando se dispara, su método de manejo se ejecuta.

11. ANEXO II: MÉTODO INVOKE

El método **Invoke** se utiliza en programación, especialmente en el contexto de eventos y delegados en lenguajes como C# (en el marco de .NET). Su función principal es invocar un método de manera segura, especialmente en situaciones donde se está trabajando con hilos y se necesita asegurar que el código se ejecute en el hilo correcto, como el hilo de la interfaz de usuario.

¿Cómo funciona?

- ✧ **Delegados:** En C#, un evento se basa en delegados, que son tipos que representan referencias a métodos. Cuando se dispara un evento, se invocan todos los métodos suscritos a ese evento a través de los delegados.
- ✧ **Invocación:** El método Invoke se usa para llamar a todos los métodos asociados a un delegado. Cuando se dispara un evento, internamente se llama a Invoke sobre el delegado correspondiente.
- ✧ **Sincronización de Hilos:** Si el evento se dispara desde un hilo que no es el hilo de la interfaz de usuario, puede ser necesario utilizar Invoke para asegurarse de que el código que manipula la interfaz de usuario se ejecute en el hilo correcto.

Ejemplo básico

```
public event EventHandler MiEvento;  
  
protected virtual void OnMiEvento()  
{  
    MiEvento?.Invoke(this, EventArgs.Empty);  
}
```

En este ejemplo:

- Se define un evento **MiEvento**.
- El método **OnMiEvento** se utiliza para invocar el evento. Utiliza el operador de null-conditional (?.) para **verificar que haya suscriptores antes de llamar a Invoke**.